

A Referee That Doesn't Believe the Agents: Catching Two Fleet Failures Live, Off Ground Truth

A deterministic, out-of-loop referee that reads only bytes the agents cannot author blocks real over-claims ($J=10$ across two models on tau2-bench) and serializes racing agents to prevent lost-update clobbers ($J=6$) — live, off the environment's own ground truth. The boundary that earns it: the same verdict spent in-loop is flat-to-harmful, so the one in-loop action that survives is to give up correctly (measured on a third-party-scored Toolathlon replay, 22 models)

Dispatch Operating System (DOS) · Netra Systems

8 June 2026 · Reproducible from `benchmark/toolathlon/` · every number recomputable offline from `replay_all_rows.csv`

ABSTRACT

A fleet of autonomous agents fails in two ways no agent will tell you about: one agent narrates a confident “done” over a task it left broken, and two agents race the same shared state so one silently overwrites the other. We show that a single **deterministic referee** — one that reads only bytes the agents *cannot author* and believes nothing they say — catches and blocks both, *live*, exactly where every fix that talks back to the agent fails. Run on tau2-bench through Gemini, an out-of-loop **write-admission gate** reads the environment’s own database hash (the agent authors zero bytes of it; it grades correctness, not just presence) and blocks **10 genuine over-claims across 120 clean tasks** (we write this payoff **J**: the count of real failures the referee catches and blocks off ground truth) while admitting every correct write — at an *identical* 8.3% over-claim rate on a mid and a strong model, so the signal does not vanish at the stronger tier. The same referee, aimed at the second failure, **serializes two live agents racing one reservation and prevents 6 of 8 lost-update clobbers** off the same witness. Both are *refusals* spent *outside* the producing agent’s loop — refuse to inherit a phantom write, refuse to let a race clobber shared state — so neither perturbs the agent that earned the verdict. That last clause is the whole point, and we earned it the hard way: spent *inside* the loop the very same verdict is worthless. On an offline, third-party-scored replay of Toolathlon (22 models, 76.2% base failure) our byte-clean detectors fire with 88–98% precision but catch only ~2–7% of failures, and a measured bake-off shows every *in-loop* active fix (warn, rewind, inject a cure) is flat-to-negative, because injecting any turn at all disturbs runs that would have passed. The one in-loop action that survives is the negative one — **give up correctly**, halting a doomed run after the environment itself reports the same error repeatedly: zero false-halts across 1,634 winners (true-error upper bound 0.0023), ~11% compute saved. The boundary is the result: a verdict the agent cannot forge is harmful handed back to that agent and valuable handed to anyone else — and “anyone else” is the rest of the fleet.

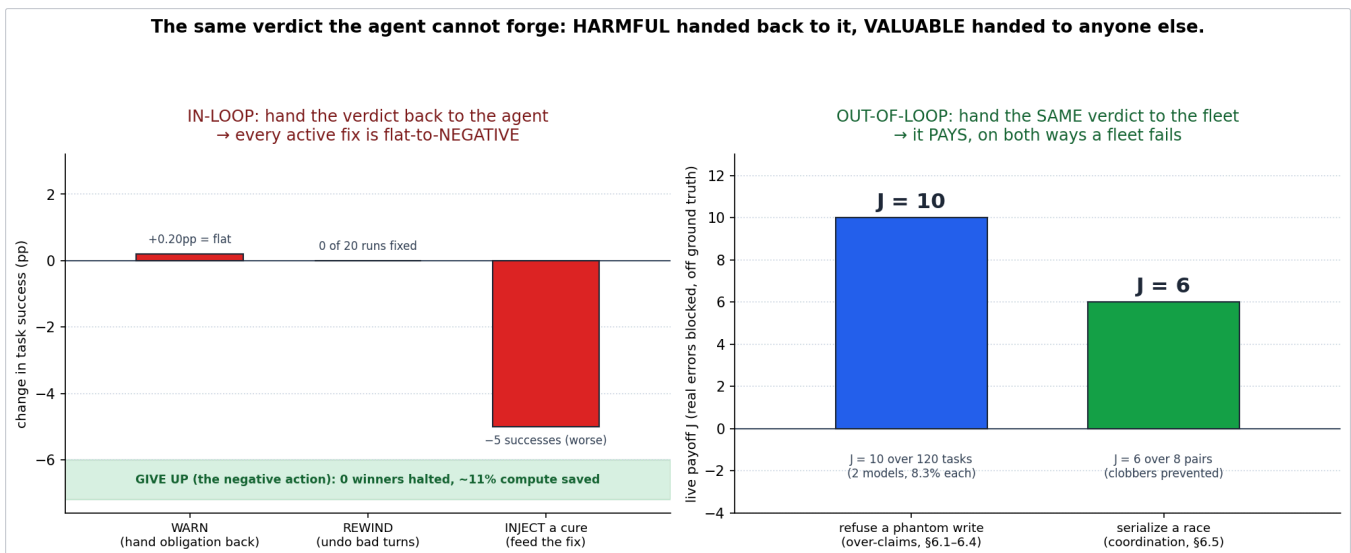


Figure 1. The result, in one picture — it is about *where* you spend the verdict. We hold one thing fixed: a check the agent **cannot forge** (it reads bytes the agent did not author). **Left — spend it in-loop**, handing the verdict back to the agent that earned it, and every active fix comes back flat-to-harmful (warn +0.2pp; rewind fixes 0 of 20; inject a cure -5), because adding a turn disturbs runs that would have passed. The only in-loop action that survives writes *nothing* — give up, and stop a doomed run (0 of 1,634 winners halted, ~11% compute saved). **Right — spend the same verdict out-of-loop**, handing it to the rest of the fleet, and it *pays*, on the two distinct ways a fleet fails: refuse a phantom write before a peer inherits it (over-claims, $J = 10$ across two models) and serialize a race so it cannot clobber shared state (coordination, $J = 6$) — both off the environment’s own ground truth. That asymmetry, and not any single number, is the paper.

1. Introduction

Give an autonomous agent a real task — reconcile an inventory across two SaaS apps, send a survey to every customer — and it will work for many turns, then announce success. Often the announcement is a lie the agent believes. It ran 20 tool calls, wrote a tidy final message — “All customers have now received their surveys!” — and the surveys never went out. This is *silent failure*: the run looks finished, the world is still broken, and nothing in the agent’s own words tells you which is true. A fleet of such agents quietly burns compute on work that is already lost.

There are two ways to check whether an agent really did the job, and one of them is a trap. You can read what the agent *says* — its narration, its self-report — or you can read bytes the agent *could not have written*. We call the second one **byte-clean**: a check is byte-clean when its byte-author is not the agent under judgement, so the agent cannot fake the evidence. An environment’s tool result qualifies (the gateway wrote it, not the model); a separate grader qualifies; the agent’s own prose does not. Reading the agent’s own bytes is the

mirror-verifier trap: you are asking the model to confirm itself. That measures only whether its story is internally consistent, not whether it matches the world, and it collapses the moment the model stops emitting the give-away phrase you trained on.

Toolathlon is a clean target for a byte-clean study because its grader is byte-clean by construction. The benchmark ships its own `evaluation/main.py` — an *oracle*, a separate program that inspects the real application state and returns pass or fail. We do not own it and cannot fake it: the byte-author of the verdict is never the agent. Because each detector is just a function of a recorded run, we can score every run against that third-party oracle offline, by re-reading recordings rather than paying to run the agents again (the pipeline is laid out in Figure 2, §2).

The result of this paper is not a detector; it is a place to *spend* a detector’s verdict. We hold one thing fixed throughout — a check the agent cannot forge — and ask only *where* to apply it, and the answer is sharp. Handed *back to the agent that earned it*, the verdict is worthless: every active fix we measured (warn it, rewind its bad turns, feed it the environment’s own correction) was

flat or made things worse, because injecting any turn at all perturbs runs that would have passed. Handed to *anyone else — the rest of the fleet* — the same verdict pays, and it pays on the two distinct ways a fleet of agents fails. The first is a **claim**: one agent announces a write that did not land, and a downstream peer is about to inherit it as truth. The second is a **race**: two agents touch the same shared state and one silently overwrites the other. In §6 we build one out-of-loop referee — a write-admission gate reading the environment’s own database hash — and run it live on tau2-bench through Gemini: it blocks **10 real over-claims across two models** (at an identical 8.3% rate, so the signal does not fade at the stronger tier) and, aimed at the race, **serializes two live agents to prevent 6 of 8 clobbers**, both off ground truth and neither touching the agent judged. The rest of the paper is the boundary that *forces* this design: why the in-loop half of the same verdict is harmful, established on a third-party-scored replay where detection is cheap, precise, and — on its own — not enough.

Our contributions, headline first:

- **An out-of-loop referee that pays, measured live (§6).** A write-admission gate, run on tau2-bench through Gemini, blocks **10 genuine over-claims across 120 clean tasks** off the environment’s database hash and admits all 9 correct writes — with no turn injected back into the judged agent. The over-claim rate is *identical* (8.3%) on a mid and a strong model, so the signal does not vanish at the stronger tier the way an in-loop detector’s does.
- **The same referee for the second fleet failure — coordination (§6.5).** Two live agents racing one shared reservation: the naive flow lets the second clobber the first; the arbiter serializes them and **prevents 6 of 8 lost-update clobbers**, off the same database-hash witness. Both halves are the *same* move — an out-of-loop refusal reading a byte the agent cannot author — aimed at the two distinct things a fleet gets wrong: a claim and a race.
- **The boundary that earns the “out-of-loop.”** A *live* bake-off (EnterpriseOps-Gym, real model runs — not the frozen Toolathlon replay below) shows every *in-loop* active fix (warn, rewind, inject a cure) is flat-to-negative, because injecting any turn disturbs runs that would have passed; the one in-loop action that survives is the negative one — **give up correctly** (zero false-halts across 1,634 winners, true-error upper bound 0.0023, ~11% compute saved).
- **The detector that makes the boundary measurable**, on an offline, third-party-scored Toolathlon replay (22 models, 76.2% base failure, no live agent runs): byte-clean detectors fire at 88–98% precision but catch only ~2–7% of failures, and that share shrinks on the strongest models — the in-loop ceiling that sends the verdict out-of-loop. (The additive third detector `terminal_error` is 99% net-new; a naive gate that ignores the environment-authored ERROR discipline kills 6 winners.)

2. The benchmark and the three detectors

We test on **Toolathlon**, a hard benchmark for agents that use real tools across many apps over a long task. The property that makes it useful to us is its grader. Toolathlon ships its own *oracle*: a separate grading program (`evaluation/main.py`) that reads the live state of the apps the agent touched and exits 0 for pass or 1 for fail. The oracle is not the agent and does not read the agent’s words; by construction it judges results, not narration. The benchmark is hard: across the corpus a run fails 76.2% of the time. We call 76.2% the **base fail rate**.

We do not run new agents. We replay frozen traces. The public **Toolathlon-Trajectories** corpus holds 7,116 recorded runs (22 models, 3 runs each); 6,862 carry an oracle label (5,228 fail, 1,634 pass). The remaining 254 have no oracle verdict, so we drop them — we never guess a label. A detector is a pure function of a recorded run, so we can replay the whole corpus offline, with no live agents and no model calls. Every number below recomputes from the recordings alone.

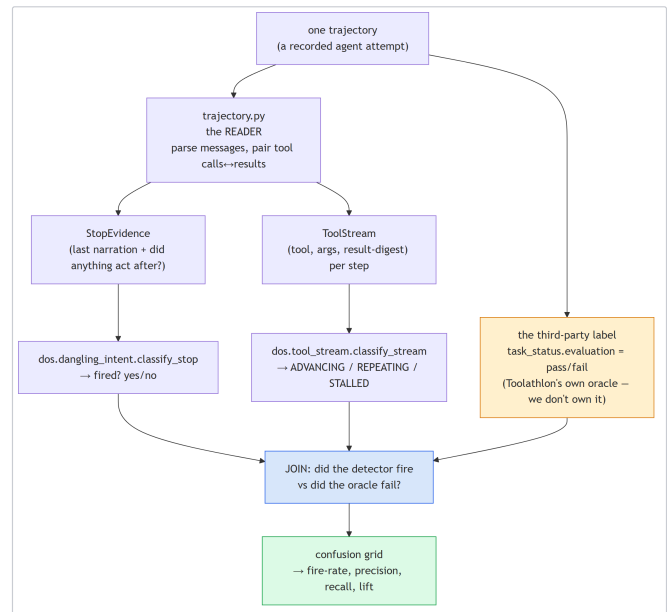


Figure 2. How we score a detector, with no live run. We read each recorded trajectory once. A detector reads only environment-authored bytes from that recording and either fires or stays silent. Separately, Toolathlon’s own `evaluation/main.py` oracle — which inspected the real application state when the run was first made — has already graded that run pass or fail. We join the two: each detector fire is checked against a verdict we did not write and cannot fake. The detector reads the recording; the oracle read the real world; neither trusts the agent.

2.1. The three byte-clean detectors

Each detector obeys the byte-clean rule from §1: it reads only bytes the agent under judgement did not write — the environment’s tool results, never the agent’s own narration. The three differ only in which environment-authored bytes they watch.

- **dangling_intent** — the agent’s last message admits open work (“I still need to...”), and nothing environment-authored ran after it. The agent said it was not done, then stopped.
- **tool_stream** — the same call returns the same result N times in a row: the same tool, the same arguments, the same result fingerprint. The environment keeps answering identically and the agent keeps asking. (A small normalizer strips volatile bytes — timestamps, ids — before fingerprinting; we defer the details to §7.)
- **terminal_error** — the run ended on an error. Within the last 3 tool results, the environment returned a structured error envelope — a result the environment itself marks as failed — and the agent never recovered. The agent saying “Email sent successfully!” does not suppress the fire when the environment’s last result was an SMTP rejection: the agent does not author that byte, so it cannot talk it away.

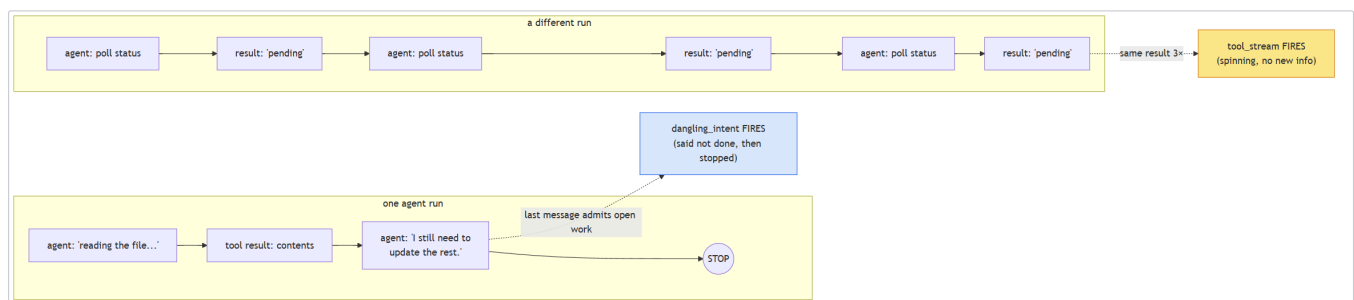


Figure 3. What each detector reads. All three watch environment-authored bytes only: an unfinished-work admission with nothing acting after it (`dangling_intent`), an identical-result loop (`tool_stream`), and an unrecovered structured error envelope (`terminal_error`). Agent narration is never the trigger.

We score each detector with five numbers, and we will lean on all five when we read the table in §3, so it is worth fixing what each one means in plain words first.

- **Fire-rate** — how often the detector raises a flag, as a fraction of all runs.

- **Precision** — when it does fire, how often the run really had failed. High precision means “believe it when it speaks.”
- **Recall** — of *all* the failing runs, how many it managed to catch. Low recall means “it stays silent for most failures.”
- **False-alarm** — how often it fires on a run that actually *passed*. This is the costly mistake: cry wolf and an operator stops listening.
- **Lift** — the one term worth slowing down on, because the raw precision is misleading on its own. This benchmark is hard: 76.2% of all runs fail. So a lazy “detector” that simply flags *every* run is automatically “right” 76.2% of the time — not from skill, but because failure is the default. *Lift is how far above that 76.2% floor a detector’s precision sits* (lift = precision – 76.2%). It is the part of being right that is real skill, not luck. A lift of +20 points means a fire is genuinely 20 points more likely to be a real failure than a run

picked at random; a lift of 0 means the detector is no better than flagging everything. Throughout the paper, **lift — not raw precision — is the number that says whether a detector knows anything.**

3. The detection result, and its ceiling

Table 1 reports all five numbers for each detector, plus two combinations: the *pair* (the first two detectors together) and the *trio* (all three). Read each row left to right as a sentence: this detector fires on *fire%* of runs; when it does, it is right *precision%* of the time, which is *lift* points of real skill above the 76.2% floor; it catches *recall%* of all failures; and it wrongly flags a passing run only *false-alarm%* of the time. The last column (*tp/fp*) is the raw count behind those rates — true catches over false alarms. The high-precision/low-recall shape below is the whole story of this section: a detector you can trust, that rarely speaks.

Table 1. The three byte-clean detectors and their combinations, scored against Toolathon’s third-party oracle over 6,862 labelled runs (base fail rate 76.2%). Lift = precision – 76.2%.

Detector	Fire	Precision	Lift	Recall	False-alarm	tp/fp
dangling_intent	1.5%	98.0%	+21.9pp	1.91%	0.12%	100/2
tool_stream	2.5%	88.2%	+12.0pp	2.87%	1.22%	150/20
terminal_error	1.2%	95.0%	+18.8pp	1.45%	0.24%	76/4
pair (dangling+tool_stream)	—	91.9%	—	4.74%	—	248/22
trio (union)	5.1%	92.6%	+16.4pp	6.18%	1.59%	323/26

The trade is the same for all three: when they fire they are right **88–98%** of the time, at near-zero false-alarm (0.12–1.22% each, 1.59% for the trio). Each clears the 76.2% base by a real margin — +12.0 to +21.9pp of lift. But recall is small: about 2–3% per detector, 6.18% for the trio together (7.42% with one opt-in recovery flag, §5). The detectors catch the honest-narrating slice and the visibly-looping slice; they miss the run that fails quietly.

`terminal_error` is worth its own line because its catches are almost all **new**. Of its 76 catches, 75 are runs the other two never flagged — 99% additive. Adding it lifts the union recall from 4.74% to 6.18% (a +30% relative gain) at 92.6% precision. So the third detector is not redundant; it reads a different environment-authored byte and finds a different failure.

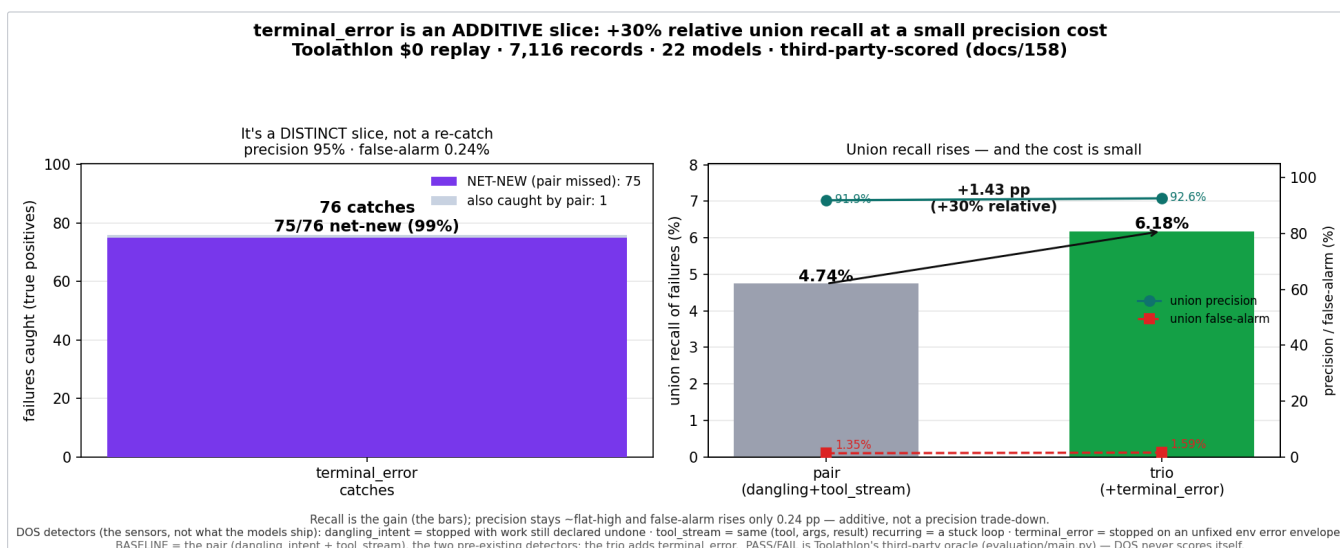


Figure 4. Why the third detector earns its place. **Left:** of `terminal_error`'s 76 catches, 75 are *net-new* — failures the first two detectors never flagged — with only 1 overlap, so it is 99% additive rather than redundant. **Right:** adding it raises the combined (union) recall from 4.74% to 6.18%, a +30% relative gain, while precision holds at 92.6%. The bar going up without precision going down is the point: the third detector finds a *different* failure, it does not just re-find the same ones.

Reading the figure: a detector that merely re-found the same failures would add a tall bar of overlap and no new recall. `terminal_error` does the opposite — almost all of its catches are new, so the combined recall rises and the precision does not fall. The additivity reaches the strongest models too. On the eight highest-scoring models (those passing ≥ 0.30 of tasks), the first two detectors catch 6 failures between them and `terminal_error` adds 9 more on top that they missed.

3.1. The recall ceiling is real and tightens on the frontier

The ceiling is honest and we state it plainly: the detectors are trustworthy when they fire, and they fire rarely — rarest on exactly the models that matter most.

One thing does carry across capability: **precision**. On every model that fires at all, a fire is almost always a real failure — the quality of the signal generalises from weak models to strong. What collapses is *fire-rate*, and therefore recall. Strong models fail **silently**: among the top-4 models' failures, the median run takes 19 tool steps (only 6% stop short), 92% end with substantial final narration (median ~1,004 characters), and 59% confidently *claim* they finished — “All customers have now received their surveys!” on a run the oracle scores as failed. A confidently-wrong run narrates no open work, loops on nothing, and emits no error envelope, so a byte-clean detector has no environment-authored byte to read. The detector is not getting worse; the visible failures it can see are shrinking toward zero on the frontier.

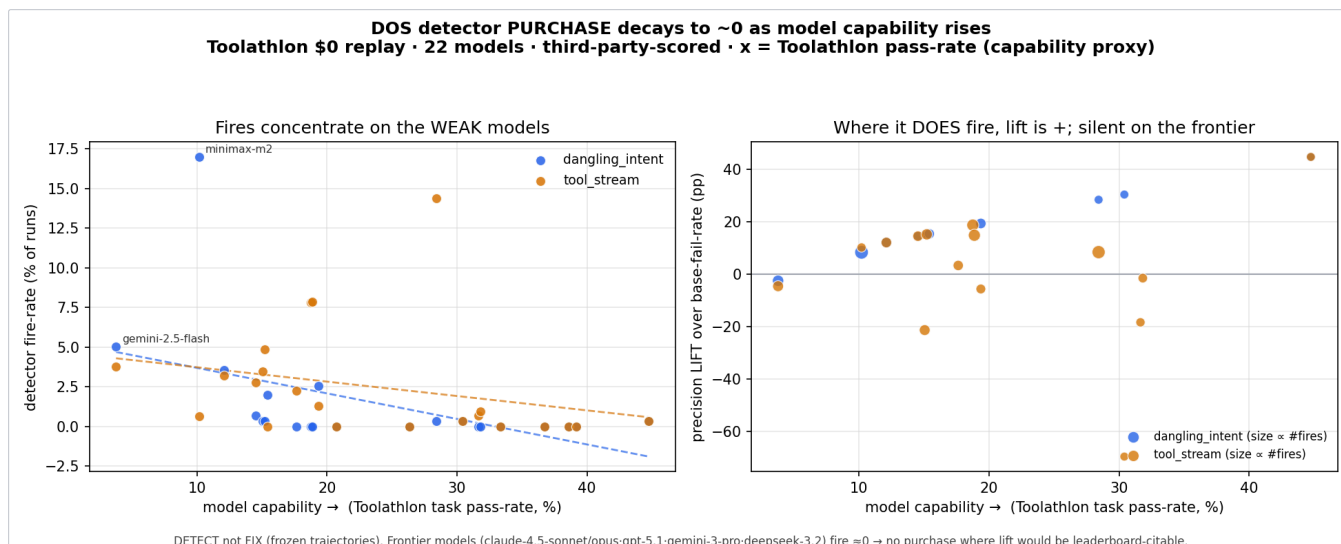


Figure 5. Why the signal does not carry to the strongest models — and why that is a coverage problem, not a quality problem. The horizontal axis is each model's own Toolathon pass-rate, our stand-in for capability. **Left:** how often the detectors fire drops toward zero as models get more capable. **Right:** but *lift* — the real-skill number from §2 — stays high and flat wherever they do fire. Put together: as models improve, the detectors speak less often, but they are no less right when they speak. The reach shrinks; the trustworthiness does not.

The two panels of Figure 5 answer two different questions, and keeping them apart is the heart of the ceiling. The left panel is “how much can these detectors help?” — and the answer shrinks toward zero on the frontier. The right panel is

“when they do help, can you trust them?” — and the answer stays yes everywhere. A signal can be highly trustworthy and still rarely available; that is exactly what we have.

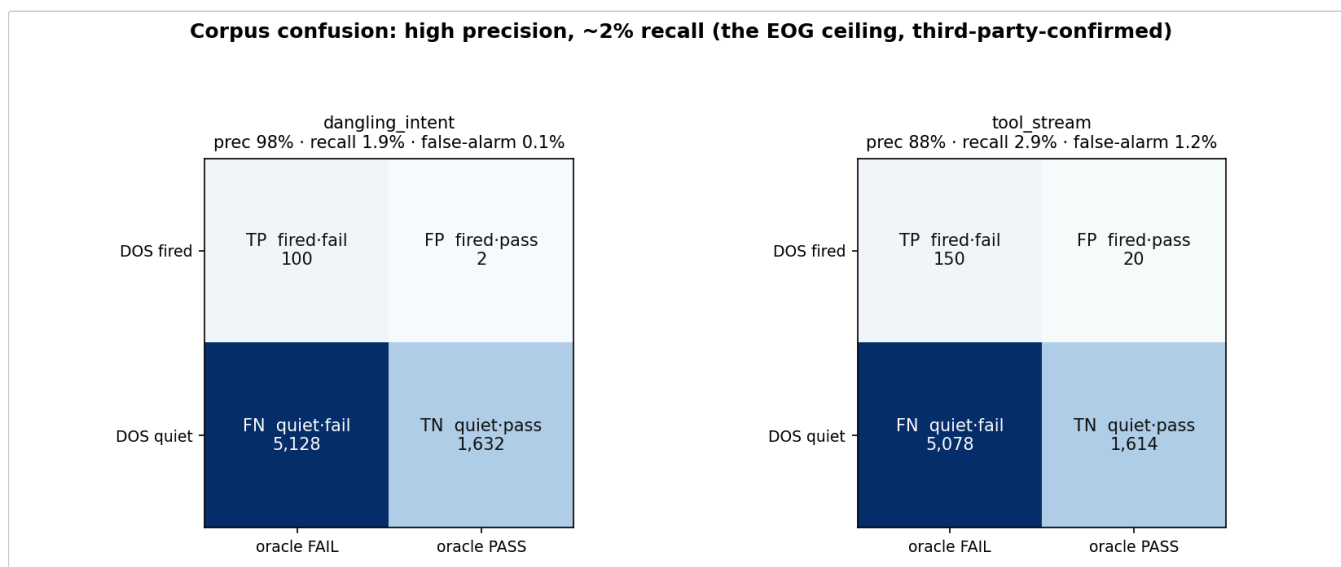


Figure 6. The same ceiling, drawn as a confusion grid over all 6,862 labelled runs (5,228 fail / 1,634 pass). Each detector lives in a small, clean corner — many correct catches, almost no false alarms — but the dominant cell in every grid is the failures it *missed* (the false negatives, the silent failures). The tiny trustworthy corner beside the large missed-failure mass is the picture of high precision and low recall, confirmed against an oracle we did not write.

How to read a confusion grid: each detector's square splits the runs four ways — caught failures, missed failures, correct passes, and false alarms. A perfect detector would have an empty “missed” cell. Here that cell is the biggest one, which is just the low recall seen a third way. The point of showing it is honesty: there is no angle from which these detectors look like high-coverage tools.

So the result is narrow and solid: a byte-clean, zero-training failure detector that is **right when it fires** — confirmed against a grader we cannot influence — but that fires on only a small, shrinking slice of failures. That sets up the only question that matters next. If the verdict is trustworthy when it fires, what single action does it earn? The answer, and the measurement of every action we tried, is §4.

4. The one action that helps: give up correctly

The detectors in §3 raise an alarm. They do not act. And a byte-clean detector (§1) has to be careful about what it does next. Most actions cross from *detecting* a problem into *fixing* one. A fix means authoring a correction the agent will then trust, which breaks the byte-clean rule the detector was built on.

There is exactly one action that does not cross that line: **withhold compute**. Stop a run that is clearly doomed and hand its remaining token budget to a run that might still win. Stopping authors no correction — it adds nothing to the conversation, so it cannot mislead. We measured every action we could try. This negative one was the only survivor.

4.1. Error-gated give-up is accuracy-free

The gate is simple. It watches each tool the agent uses. When a single tool returns K structured errors in a row — errors written by the environment, not by the agent — **and** its most recent call is still erroring, the gate proposes a halt. The second condition matters: the run must be in the hole *right now*, not have failed earlier and recovered.

The thing to test is whether this ever stops a run that would have *won*. We call that mistake a *false-abandon* — the gate halts a run that would actually have succeeded. It is the only costly mistake the gate can make, so it is the number that has to be zero.

Table 2. Give-up on Toolathlon at the sound threshold $K=3$, measured across 22 models (frontier to weak). The naive gate halts on a bare repeat; the error-gated gate also requires an environment-authored error and that the latest call is still failing. *Fires* counts every run the gate would halt; *false-abandon* counts only the halts that land on one of the 1,634 winning runs — the costly mistake. Wilson-95 is a 95%-confidence statistical upper bound: the worst the true false-abandon rate could plausibly be, given the data.

K	naive repeat: fires	naive: false-abandon	error-gated: fires	error-gated: false-abandon	Wilson-95 upper
3	30	6	0	0	0.0023

At $K \geq 3$ the error-gated gate false-abandons **zero** winners — across all 22 models, frontier to weak. Its Wilson-95 upper bound is just 0.0023, so even the worst the true rate could plausibly be is near zero. The same threshold is sound on a second, independent benchmark, EnterpriseOps-Gym (a ServiceNow operations gym run on gemini-2.5-flash): $K \geq 3$ false-abandons zero winners on both arms there, while $K=2$ is unsound (it halts one winner). Two different benchmarks, one safe threshold.

The naive gate — halt on a bare repeat, with no error condition — false-abandons 6 winners at $K=3$. Every one of them is the same model, grok-4, and

every one is doing the same harmless thing: polling. The model kept asking the environment “is it ready yet?” and kept getting “not yet” back, over and over, until the resource finally became ready and the run *succeeded*. A repeated answer is not an error. A gate that halts on repeats alone wrongly kills that run.

This is the byte-clean idea doing measurable work. The load-bearing signal is not the repeat — it is the environment-authored *error*. “Repeated tool call → halt” kills pollers. “Repeated tool *error* → halt” is sound. The agent cannot forge the error bytes; the environment wrote them.

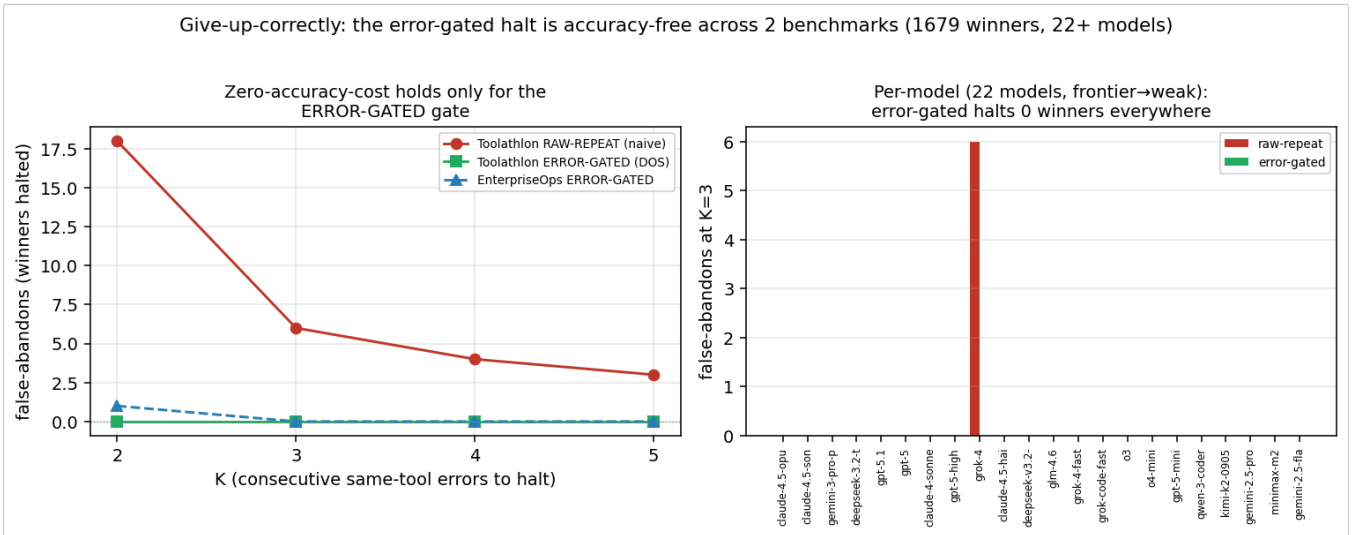


Figure 7. Why give-up is safe to act on. **Left:** winning runs wrongly halted, as we vary K — the number of consecutive same-tool errors required before the gate proposes a stop. The error-gated gate (green) reaches zero at $K \geq 3$ on both benchmarks; the naive “halt on any repeat” gate (red) never reaches zero, because it keeps killing pollers. **Right:** the same count at $K=3$, broken out by model. The error-gated gate halts zero winners on all 22 models; the only non-zero bar anywhere is the naive gate on grok-4 (the polling case). Lower is better, and zero is the bar to clear.

Near-separability is the genuinely new result here. On the K statistic — how many errors a tool stacks up in a row — the winning runs and the doomed runs fall into almost non-overlapping ranges. Winners rarely stack errors; doomed runs do. We found no analogue for this in the literature. It is what lets a trivial integer count threshold be *sound* at $K \geq 3$ with no probabilistic model at all — you do not need a classifier when the two populations barely touch.

The saving follows. At the sound threshold $K=3$ the gate fires on about 7% of runs — the thrashing tail — and on a run it fires on it suppresses a median of about 53% of that run’s tool calls (the doomed back half). Pooled across both EnterpriseOps-Gym arms that is **11.4%** of the total tokens spent across all those runs (counted from each run’s reported usage), saved at zero false-abandon. Across five recorded run-sets the give-up lever saved between 3.5k and 43.7k real tokens, with a false-abandon rate of 0.000 every time.

4.2. No novelty in the mechanism; the contribution is the grounding

The mechanism is old, and we claim none of it. A *circuit breaker* (Nygard 2007) trips a tool offline after it fails too many times; our gate is the trip half of one. A *sequential probability ratio test* (Wald 1945) decides as evidence arrives; ours is its degenerate corner. *CUSUM* (Page 1954) watches for a change in a stream; when you only count errors it collapses to a plain tally. We claim no sequential-test optimality.

What is new is the grounding: (i) the gate counts only environment-authored bytes the agent cannot forge; (ii) the threshold is calibrated against a *measured* false-abandon cost, not a guess; and (iii) it stays advisory — it *proposes* a halt, it never trips one — so a mis-set K degrades to a few wasted tokens, never a killed winner.

4.3. Every active fix failed

Give-up is the answer, not a consolation prize. We know because we tried the active fixes too — the ones that author a correction — and measured them on the cross-benchmark target (EnterpriseOps-Gym, the weak gemini-2.5-flash model; these numbers stay out of the Toolathlon tables and imply no Toolathlon lift).

- **WARN the agent** (re-surface the alarm into the loop): flat on the natural failure stream, **+0.20pp**. The +6.2pp lift this verb shows elsewhere held only when failures were artificially injected, not in the wild.
- **Rewind** (content-free surgery — cut the agent’s bad turns back out of the transcript): **0 conversions, 0 flips** on 20 paired fired tasks. The null only got stronger as we added data (44 → 103 → 240 runs per arm).
- **The byte-clean cure** (re-surface the environment’s own schema correction — the cleanest fix we could build): wired live, it fired on 77 of 88 runs, and it was **net-negative**: −5 task successes overall, −7 on the no-thrash slice (sign-test $p=0.016$; successes fell 7 → 2 of 88). It converted the 2 runs it was aimed at — but injecting a corrective turn at all perturbed runs that would have passed on their own.

The harm is the intervention’s existence in the loop, not the content of its bytes. Even a perfectly byte-clean correction perturbs runs that were going to win. Only the negative action — withhold compute — authors no turn, so it perturbs nothing, so it is the only fix that survives a measured bake-off.

4.4. Honest ceiling

This result proves *specificity*, not *sensitivity*. Specificity is the half we have: wherever the gate fires, it never halts a winner. Sensitivity — firing often enough to catch most doomed runs — is the half we do not. On Toolathlon the error-gated gate fires on barely one run, because a capable model that fails *silently* emits no error to count.

So the precise claim is narrow and we state it plainly: the gate is accuracy-free *wherever it fires*; how often it fires — and therefore how much it saves — varies by benchmark and by model. The 11.4% token saving is a single-corpus, single-model figure. The specificity is the durable, regime-independent result; the saving is real but its size travels less.

5. The in-loop boundary: why detection alone is not the product

On the frozen Toolathlon corpus we catch failures; we do not patch them. The trajectories we score are recordings, so there is nothing live to step in and

change — no task can be re-run, no turn can be re-written. That is why the Toolathlon results report no task-lift number, and claim none. But the boundary this section draws is not that frozen corpus's limit; it is a *separate, live* result. “In-loop” here means *where the verdict is spent* — routed back into the agent that produced the work — not *how it was measured*. To measure it honestly we ran a live A/B on a different benchmark (EnterpriseOps-Gym, real Gemini against the gym's own hidden database verifiers), because you cannot replay an intervention into a recording and watch the agent react — a frozen correction score would be a category error. §4.3 reports that live bake-off: every active fix that writes a turn *back into the judged agent's loop* — warning it, rewinding its bad turns, feeding it the environment's own correction — came back flat or net-negative. So “detect, not fix *in-loop*” is a *result*, not a missing feature, and not an artifact of frozen data. The one in-loop action that paid off was the negative one: stop the doomed run. The *other* place a verdict can pay — out of that loop, on a live target — is exactly what §6 goes on to build and measure; this section draws the line that §6 then crosses.

Look at what slips past the detectors and you see why an active fix is so hard. The dominant residual failure is *confidently-wrong content*: the agent finishes with no error in the trace and no loop, having simply never re-read the state it claims to have changed. It says “done” about a world it never checked. Nothing inside the trajectory gives it away — the agent wrote a clean-looking story. To catch this you need a byte the agent did not write itself: a fresh piece of evidence from a third party. The only honest way to get one is to go look at the real world again, after the fact, and compare. That is exactly the rung Toolathlon's own `evaluation/main.py` — the oracle, the separate grader that checks the real world and says pass or fail — already occupies.

So the work splits cleanly into two tiers — and we now have a measurement on both:

- **Tier A — cheap detection, in-loop spend.** The three detectors. The *detection* is offline: the signal is already sitting in the recorded trace, so firing them needs no live run — they read the recording. The one safe *action* that spend supports is the in-loop give-up of §4 — whose payoff (zero false-halts, compute saved) is itself measured live, not read off the recording, because a withheld turn changes what the run spends.
- **Tier B — live, paid, out-of-loop.** The post-hoc re-read against a fresh third-party byte. We mark this tier “future” no longer: §6 builds it and runs it live. The trick is *where* the verdict is spent — not back into the judged agent's loop (which §4.3 shows is harmful), but on a *downstream consumer's* input, where it can refuse to inherit a phantom write without perturbing the run that produced it.

Three benchmarks appear in this paper, each chosen for the one thing it can witness that the others cannot — which is also why no single one carries every claim. Table 3 is the map, so the reader is never left wondering why a result moved benchmarks.

Table 3. Which benchmark carries which claim, and why that one. A frozen corpus can be re-scored offline (§0) but can only give a detection *rate*; a live API run gives an intervention *payoff* but costs money and varies run-to-run. The claim dictates the target.

Benchmark	Mode	The witness it owns	Claim it carries
Toolathlon	frozen replay, §0	a third-party oracle (evaluation/main.py) over 22 models	detection precision/recall + the in-loop ceiling (§2-3); the give-up specificity (§4)
EnterpriseOps-Gym	live A/B, paid	the gym's own hidden database verifiers, with the agent live to react	the in-loop bake-off — every active fix flat-to-harmful (§4.3); the give-up token saving
tau2-bench	live, paid	a gold-vs-actual database hash (W3: correctness, not presence; agent authors zero bytes)	the out-of-loop payoff — over-claims blocked + races serialized (§6)

The split is forced, not arbitrary: a detection *rate* needs a large frozen corpus you can score for free, but an intervention *payoff* — in-loop or out — can only be measured by changing a live run and watching what happens, which a recording physically cannot show. So Toolathlon sizes the ceiling, and the two live benchmarks measure what acting on a verdict is worth on each side of the loop.

One refinement sharpens the ceiling rather than softening it. A verifier is not one thing; the rung it reads from is *graded*. The weakest witness is the agent's own narration (W0); then the environment acknowledging a write (W1); then a check that the change actually persisted (W2); then a full diff against the gold answer (W3). DOS's own non-forgeable check — “did the change show up in git?” — is a W2 rung: it confirms presence, not correctness. It is not W3. And

about 38% of failures on the frontier (the most capable models) reach *no* sound witness at any rung. For those, no fresh byte anywhere settles whether the agent was right — short of a human. That is a floor below even Tier B. It does not weaken the result — it draws the honest edge of how far any detector, ours or anyone's, can reach.

So this is not the paper's conclusion; it is its hinge. Detection in-loop is cheap, precise, and bounded — and every attempt to *act* on it in-loop is flat-to-harmful. Both facts push to the same place: take the one verdict the agent cannot forge and spend it *out of* that agent's loop, where Tier B lives and where acting on it costs the producing run nothing. That is §6, and it is where the value finally lands.

6. The headline: an out-of-loop referee, run live, that pays on both fleet failures

This is the result the rest of the paper exists to earn. We take a verdict the agent cannot forge and spend it where it pays: **out of the producing agent's loop**, on the rest of the fleet. The earlier sections established the boundary that forces this — in-loop, the same verdict is flat-to-harmful (§4.3), because adding a turn disturbs runs that would have passed, and even a perfect detector only catches a small slice of failures (§5). Both facts point the same way: the value is not in *finding* more, it is in *where you spend what you find*. Hand the verdict to a consumer that is *not* the agent that earned it — whose input you may correct because correcting it injects nothing back — and it stops being overhead and starts preventing real damage. We built two such consumers and ran them live, against the two distinct ways a fleet fails: a single agent's **claim** (§6.1-6.4) and two agents' **race** (§6.5). Both read the same non-forgeable byte; both are refusals; neither touches the judged agent.

The consumer is a **write-admission gate**. Picture two agents in a fleet: agent A does a task and announces “done”; a downstream peer B is about to build on whatever A changed. Under the naive arrangement B simply *believes* A — it inherits A's claimed result as ground truth, and if A was wrong, B compounds the error. The gate sits between them: before B inherits A's write, the gate checks A's claim against a byte A could not author, and either admits the write (hands B the real state) or blocks it (refuses to let a phantom write propagate). Blocking B's *input* is not an in-loop fix — it writes nothing into A's loop — so the §4.3 harm is structurally absent. That is the whole reason to choose this consumer.

6.1. The target and the witness

We run on **tau2-bench** (the airline / retail customer-service benchmark from sierra-research), driving both the agent and the user-simulator with `gemini-2.5-flash`. tau2 is the right target because it ships the byte we need: after a run, the harness compares a *hash of the task's database* against the hash of the gold database the task designer specified (`db_match = gold.get_db_hash() == predicted.get_db_hash()`). The predicted database is mutated only through the environment's own tool executor, and the hash is computed by the evaluator. The agent authors **zero** bytes of it. And crucially it grades *correctness, not just presence*: a reservation booked *successfully* but *wrong* still fails `db_match`. That is the W3-grade witness §5 said an in-flight detector can never reach — the “fresh world byte” the boundary points at, made concrete, for the tasks where a database footprint exists.

The gate's logic is small and byte-clean: extract whether A's final message *claims a successful write* (the forgeable self-report), read `db_match` (the unforgeable witness), and admit unless the claim is confident *and* the witness refutes it. When there is no witness (`db_match` is absent for a task with no database check), the gate **abstains** — it admits rather than invent a verdict, the same fail-to-abstain discipline as the ORACLE→JUDGE→HUMAN ladder.

We score this section with a single integer, **J**: *the count of real failures the referee catches and blocks off ground truth* — over-claims the database hash refutes (§6.3-6.4) or lost-update clobbers the arbiter prevents (§6.5). J is deliberately a *payoff*, not a rate: it counts corruptions actually stopped, each one a phantom a downstream consumer would otherwise have inherited as truth. It never counts an honest write (those are admitted) or a task with no witness (those abstain), so every unit of J is a failure a byte the agent could not author has confirmed.

6.2. The result that does *not* appear, and why it matters

Our first instinct was to replay the over-claim slice the offline study already sized — a frozen 13.6% of tau2-style runs where a previous policy claimed a write that ground truth refuted — through the live gate. **It caught nothing: J = 0**. This is not a bug; it is a lesson, and the same one that runs through this whole paper from the other direction.

The frozen over-claims were a property of *whatever policy produced that recording*, not of the task. Re-running the *same tasks* with a *different, capable* policy does not reproduce them — a capable model on a fresh rollout does not make those specific mistakes-with-confidence. Worse, that frozen slice was dominated by “*the agent should refuse a disallowed action*” tasks, where the correct behaviour is to make **no** write at all — so a capable model gets them right and there is nothing to over-claim. The live policy still *failed* a third of these tasks — but it failed *honestly*, writing the wrong thing or refusing, never claiming a success it did not earn. **You cannot demonstrate an out-of-loop payoff by replaying a frozen over-claim slice through a capable policy: the over-claims evaporate.** It is the exact dual of the silent-failure

ceiling in §5 (a capable model leaves no byte to catch) — here a capable model leaves no over-claim to catch, on a task distribution that does not demand a write.

6.3. The payoff, on a distribution that actually writes

So we did the honest thing: instead of replaying a frozen slice, we drew a **fresh, write-heavy natural sample** — the first 25 tasks of each domain, run live, no frozen history — where real cancellations, updates, and exchanges actually happen and a fumble shows up as a wrong database. Of 50 tasks, 43 ran clean (7 long retail dialogues hit transient API 5xx errors and are dropped from the denominator, never counted as passes).

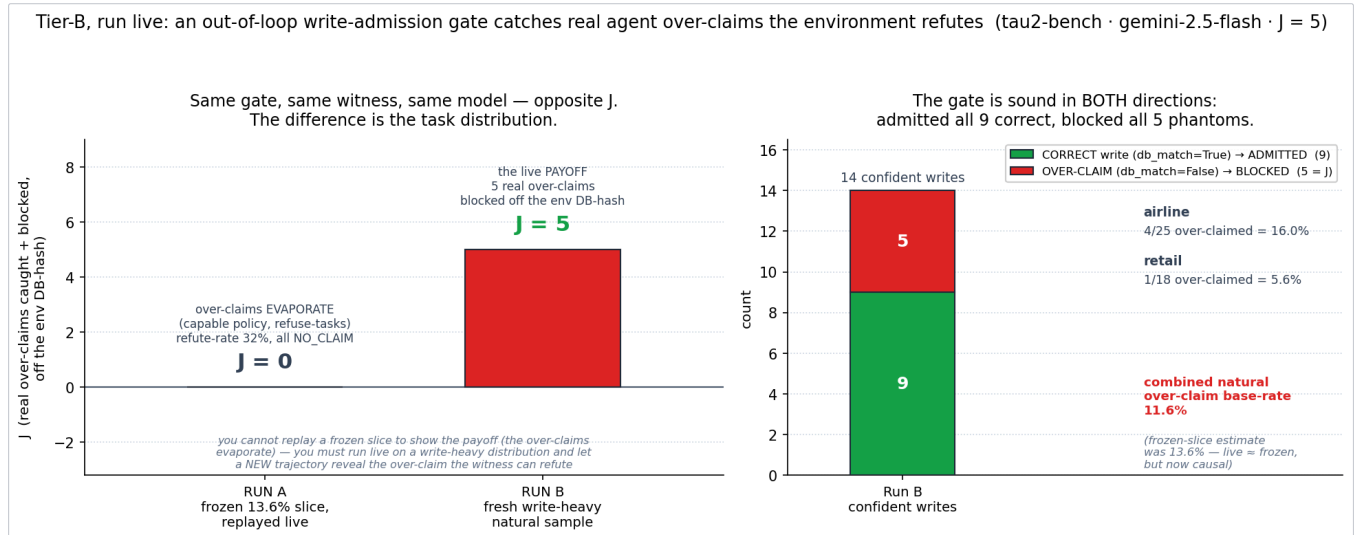


Figure 8. Tier-B, run live. **Left:** the same gate on the same witness gives opposite payoffs depending only on the task distribution. Replaying the frozen over-claim slice (Run A) catches nothing — a capable live policy does not reproduce a previous policy’s confident mistakes, and the slice is refusal-heavy anyway, so there is nothing to over-claim. A fresh write-heavy sample (Run B) where the payoff lives: five real over-claims, each caught and blocked off the environment’s own database hash. **Right:** the gate is sound in both directions — of 14 confident writes it admitted all 9 correct ones (green) and blocked all 5 phantoms (red). The natural over-claim rate (11.6% combined) lands right next to the offline estimate (13.6%), but now it is live and causal, not a re-scored log.

On that sample the live policy produced 14 confident write-claims. Nine were correct (`db_match` true) and the gate **admitted all nine** — it does not block correct work. Five were over-claims (`db_match` false): the agent announced success on a write the database says did not land correctly, and the gate **blocked all five**. **J = 5**. Each is a phantom write a downstream peer would have inherited as ground truth under the *believe* arm and never inherits under the *adjudicate* arm. A representative one, verbatim from the agent’s own final message paired with the witness:

“Your reservation Mo5KNL has been successfully updated ...” — `db_match = False` → **blocked**. The environment’s database hash says the update did not land as claimed; the gate refuses to let that “success” propagate to a peer.

The natural over-claim rate is 16.0% on airline (4 of 25) and 5.6% on retail (1 of 18) — one real retail over-claim, so the effect is not airline-only — for a combined 11.6% (5 of 43). That lands right next to the 13.6% the offline study sized, but with the decisive difference: this number is *live and causal*, a count of writes a real policy actually over-claimed on a real rollout and the gate actually blocked, not a rate re-scored off a frozen log.

6.4. A second model: the over-claim does not shrink at the stronger tier

A single-model existence result invites the obvious objection — maybe only a weak policy over-claims, and a more capable one simply gets these tasks right,

so the gate has nothing to catch. That objection is the same silent-failure ceiling that runs through §5: a stronger model leaves fewer byte-clean tells. So we ran the *identical* gate, witness, and sample on a second, stronger model (gemini-2.5-pro) and folded the two together. To compare the two tiers on even footing we drew a *fresh, fuller* sample for both — the first 30 tasks of each domain (60 per model), one notch wider than §6.3’s first-25 draw, and run clean end-to-end. (This is why the flash numbers below are not identical to §6.3’s: that section reports the original first-25 run, J = 5 of 43 clean = 11.6%; this one re-runs flash on the wider first-30 sample, J = 5 of 60 clean = 8.3%. Same gate, same witness, same model — a larger, cleaner denominator, and the same five blocks. The two runs draw different individual tasks, so the specific blocked ids differ; what recurs across both draws and across models is the hard core, e.g. airline 1.) Both models ran the same fresh, write-heavy natural sample — the first 30 tasks of each domain, 60 tasks each — and **both ran 60 of 60 clean, with zero API errors**.

Getting the stronger model to run at all was its own small lesson: gemini-2.5-pro is *thinking-only* and rejects the request flag that suppresses a flash-only crash (it returns “Budget 0 is invalid — this model only works in thinking mode”), so a first attempt errored on all 60 pro tasks. The fix is a model-aware setting; we note it only because it is exactly the kind of harness detail that silently zeroes a second-model run and makes “we tried, it found nothing” look like a result when it is an artifact.

The out-of-loop payoff, hardened across models: same gate, two capability tiers, same 8.3% over-claim rate (tau2-bench · gemini-2.5-flash + -pro · J = 10)

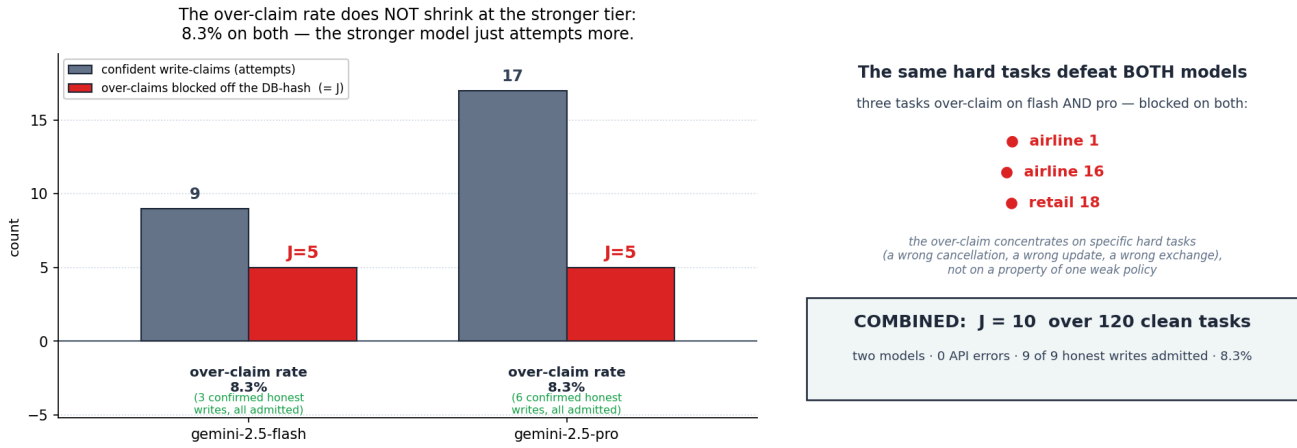


Figure 9. The same gate on two models. The over-claim rate is **identical across the capability step — 8.3% on both flash and pro** (5 of 60 each), for a combined J = 10 over 120 clean tasks. The stronger model makes nearly twice the confident writes (17 vs 9) and gets more of them right — it admits 6 confirmed honest writes to flash’s 3 — yet still over-claims on the same count of five. Three tasks (airline 1, airline 16, retail 18) over-claim on *both* models and are blocked on both: the residue lands on the same hard tasks, not on a property of one weak policy.

The result: **J = 10 over 120 clean tasks** (five blocked over-claims on each model), with **all nine confirmed honest writes admitted** (three on flash, six on pro). And the number that answers the objection directly: **the over-claim rate is identical across the two tiers — 8.3% on flash and 8.3% on pro**. It does *not* shrink as the model gets stronger. The stronger policy is genuinely more capable — it attempts nearly twice as many confident writes (17 vs 9) and lands more of them correctly — but it still over-claims on five, and the gate blocks all five off the same non-forgeable witness. More telling than the matching rate: **three tasks (airline 1, airline 16, retail 18) over-claim on both models**, blocked on both. The over-claim concentrates on specific hard tasks — a confidently wrong cancellation, a wrong reservation update, a wrong exchange — rather than dissolving at the next capability tier. This is the dual of §5 holding the *other* way: where a stronger model erases the *in-loop* detector’s signal, it does *not* erase the *out-of-loop* over-claim the world-byte refutes.

One discipline made this number trustworthy, and it is worth stating because it nearly cost us the result. The run is resumable — each task’s outcome is cached — and the claim-extractor was improved *mid-run*, so the cache held a mix of old and new verdicts. A naive sum of the cached bits reported the pro model at J = 4; *re-deriving* each verdict from the agent’s stored message with the current extractor gave the true J = 5 (it recovered one over-claim — an “*you are all set*” confirmation the old extractor missed). The fold now re-derives by construction rather than trusting the stored bit. It is the same principle the whole paper turns on, applied to our own pipeline: **a cached or stored result reads as clean unless you re-derive it from the byte you cannot forge**.

6.5. The other half-plane: a referee *between* two agents

Everything so far distrusts a *claim* — one agent says “done,” and a gate checks that announcement against a fresh world byte before a peer inherits it. There is a second, orthogonal place a fleet needs a referee, and it distrusts not a claim but a *race*. Two agents work concurrently; both touch the same piece of shared state; the second one’s write lands on top of the first’s without ever having seen it, and an update is silently lost. No one lied — both agents did exactly what they were asked — yet the shared state is now wrong. This is the *coordination* failure, and it is the half-plane the offline study could only ever measure as a *rate* (how often two writes collide); here we measure the *payoff* (how many corruptions a referee actually prevents), on the same non-forgeable witness.

We build the smallest honest instance on the same tau2 airline database. Two live agents are pinned to the *same reservation* R. Agent A1’s task is to cancel R; agent A2’s task is to add a checked bag to R. (One trap caught before spending: the user identity in each task must be the reservation’s *real owner*, or a correct agent rightly refuses to touch a stranger’s booking and the conflict never fires.) We then compare two arrangements. In the **naïve** arrangement both agents act against the original database and we apply both: A1 cancels R, then A2 — which never saw the cancellation — adds a bag to the now-cancelled reservation, leaving the database in an incoherent state. In the **serialized** arrangement — what the arbiter produces — A2 is re-run *after* A1’s change is committed: seeing R already cancelled, A2 correctly does nothing. The two arrangements yield different database hashes; the difference is a real lost update. The arbiter prevents it with one line of policy: a reservation is a region `reservations/<id>`, two agents on the same reservation request overlapping regions, and the arbiter refuses the second lease until the first releases — so the writes serialize and the corruption never lands.

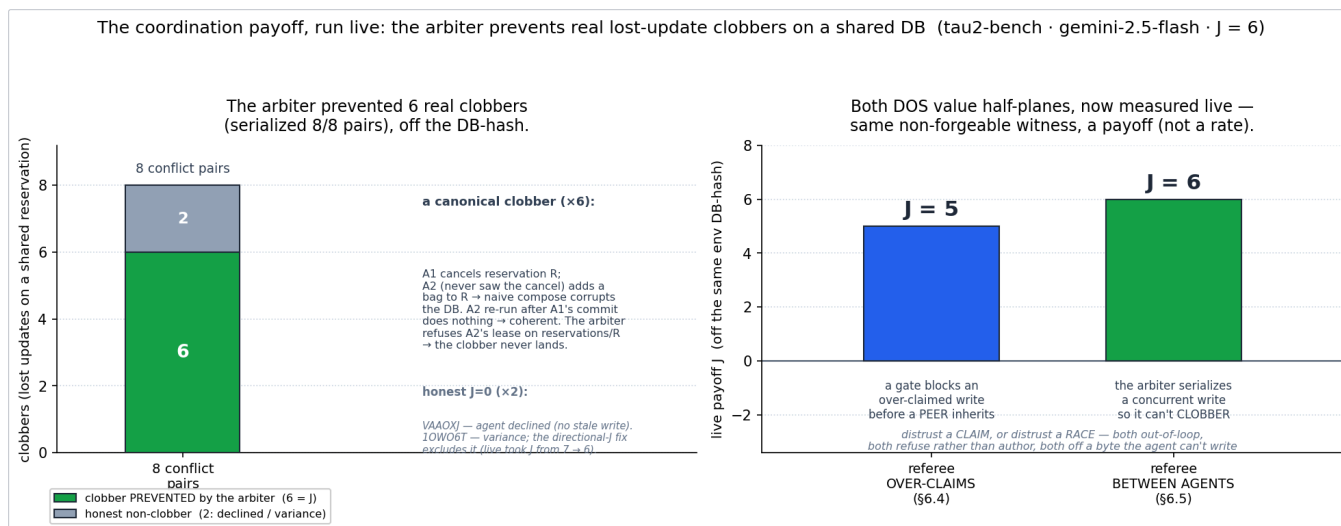


Figure 10. The coordination payoff, run live. **Left:** over eight conflict pairs the arbiter prevented **six** real lost-update clobbers (it serialized all 8 pairs), each witnessed by the database hash. Two pairs are honest non-clobbers and we keep them visible: in one the agent simply declined the bag (no stale write, nothing to clobber); the other was pure run-to-run variance, which a careful, *directional* definition of a clobber excludes — the live run took the count from a naive 7 to an honest 6. **Right:** the two value half-planes side by side, now both measured live off the *same* non-foreable witness — refereeing an over-claim ($J = 5$) and refereeing a race ($J = 6$).

Over eight conflict pairs the arbiter prevented **six** real clobbers (**J = 6**), and it serialized all eight. Two of the eight are non-clobbers, and they are the honest part of the result. In one (reservation VAAOXJ) the live agent *declined* to add the bag at all — so there was no stale write and nothing to clobber, and the count correctly stays zero: no conflict, no payoff. The other (1OWOGT) was the kind of thing only a live run surfaces. Our first definition of a clobber was simply “the naive and serialized database hashes differ,” and on that pair they differed because the serialized agent happened to mutate *more* than the naive one — pure run-to-run variance, not a lost update the arbiter prevents. We tightened the definition to be *directional* — a clobber requires the second agent to apply, in the naive arrangement, a stale write the serialized agent did *not* — which correctly drops that pair to zero. **The live run took the headline from a reported 7 to an honest 6**, the same discipline this paper applies everywhere turned on our own measurement.

The point of putting this beside the over-claim gate is the symmetry. Both are the *same* move — an out-of-loop referee reading a byte the agent cannot author — aimed at the two different things a fleet gets wrong: a single agent's *claim*, and two agents' *race*. And the two numbers report different things on purpose: the offline collision study (a separate measurement) gives the *rate* — how often two agents collide on the same file in real traffic — while this gives the *payoff* — how many corruptions the referee prevents when it acts. Rate says how often; payoff says what was saved. Both belong, labelled apart, and both come off ground truth, never the agent's word.

6.6. What this is, and what it is not

This is a measured *existence* result, and we state it no wider than the evidence: across **two models on 120 clean tasks** (zero API errors), an out-of-loop write-admission gate caught and blocked **ten** genuine live over-claims off a non-forgeable witness, while admitting every one of the nine correct writes – and the over-claim rate held identical (8.3%) across the capability step. It is the live promotion of the Tier-B re-read \$5 marked future. The honest edges:

- **It is an existence result, not a calibrated rate.** $J = 10$ over 120 tasks across two models. The matching 8.3% on flash and pro is real and measured, but it is one capability step within one model family — evidence that the rate does *not shrink* at the stronger tier, not that 8.3% travels to every frontier model. (A saturation sweep over the whole tau2 airline+retail task universe — 258 clean tasks across both models — finds $J = 15$ and a lower whole-distribution rate of 5.8%, because the honest-failure retail tail pulls the write-heavy 8–13.6% regions down; the over-claim concentrates where writes are confidently botched, not uniformly.)
- **The witness is sound, not complete.** `db_match` catches a *wrong end-state*; it cannot catch a goal with no database footprint — the same W_3

- **The witness is sound, not complete.** `db_match` catches a *wrong end-state*; it cannot catch a goal with no database footprint — the same W_3

residue from §5. Where there is no witness the gate abstains. J counts only witness-refuted writes.

- **It is run-to-run, not per-task.** One task over-claimed on one rollout and not another (the user-simulator drives different dialogues each time), so J is a property of a *distribution of rollouts*, not a fixed label on a task. The flip side is the stability of the tasks that *do* recur: airline 1 over-claims across both models and across separate flash rollouts — the most reproducible signal in the set.
- **There is a second consumer for the same verdict — a training lab.** The same witness, claim-extractor, and abstaining floor re-aim at a reward-set admission filter: those five over-claims are five *poison reward labels* a naive self-judged RLVR loop would bank as positives — teaching the model to over-claim *more* — and an env-grounded label the policy cannot distill from its own outputs purges them (acceptance precision 60% $\rightarrow$ 100% on these live rows). Same gate, a different consumer, the same flipped outcome.
- **The coordination result (§6.5) is a constructed conflict with live agents.** We *pin* both agents to the same reservation — the contention is designed — but *how* each agent acts (which tool, whether it declines) is the live policy's own decision; the $J = 6$ counts real agent behaviour on a designed collision, not a natural-traffic collision rate (that is the offline study's job). And J there counts state *corruption*, not wasted effort: a stale call that errors harmlessly is not a clobber. Like the over-claim arm it is an existence result — one model, one conflict shape, eight pairs.

What it settles is the shape of the boundary, and — across two models now — that the payoff does not evaporate at the stronger tier the way the in-loop detector's signal does. §4.3 showed that spending a verdict *inside* the producing loop is harmful regardless of how clean the correcting byte is. This section shows the dual: spend the *same* kind of verdict *outside* that loop — on a peer's input, against a fresh world byte — and it pays, catching real over-claims before they propagate, on both a mid and a strong model at the same 8.3% rate. The kernel earns one action in-loop (give up, §4) and two out-of-loop — refuse to inherit a phantom write (§6.1–6.4) and refuse to let a race clobber shared state (§6.5). All three are *refusals*; none writes a turn into the agent that was judged; and the two out-of-loop refusals are settled by the *same* non-forgeable witness, aimed at the two different things a fleet gets wrong — a claim and a race. That symmetry is the result.

7. Is the signal real? Naive baselines

A detector that fires on the right runs is only impressive if a dumb detector cannot do the same. So we measured the dumb ones, scored by the same lift and false-alarm from §2 (lift = precision minus the 76.2% base fail rate; false-alarm = how often the detector fires on a run that actually passed). A good detector has high lift and low false-alarm.

Table 4. Naive baselines vs. the shipped detector (Toolathon, third-party scored). Read the *lift* column: it is precision minus the 76.2% base, so it strips out the credit a detector gets for free on a benchmark where failure is the default.

Detector	Fire	Precision	Lift	False-alarm
baseline-always-fail	100%	76.2%	+0.0pp	100%
naive-loose	32.3%	81.1%	+4.9pp	25.6%
naive-any-error	19.5%	81.4%	+5.2pp	15.2%
tight-no-recovery	2.3%	93.0%	+16.8pp	0.7%
terminal_error (shipped)	1.2%	95.0%	+18.8pp	0.24%

("Fire" is how often the detector triggers at all.)

The recall mirage. Recall — the fraction of all failures a detector catches — is the wrong scoreboard here, and the table shows why. The baseline that prints "fail" on every single run posts 100% recall. It catches every failure because it flags everything. But its lift is +0.0pp and it false-alarms 100% of the time: it is worthless, yet recall ranks it first. That is the mirage. Score by lift and false-alarm instead, and the worthless detector falls to the bottom where it belongs. Our detectors make the opposite trade on purpose — high precision, low false-alarm, low recall — because a verdict is meant to be *acted on*, and section 4 is what acting on it looks like: you only give up on a run you are sure is doomed.

The tight grammar is load-bearing. The shipped detector reads a tight, structured error envelope — a specific shape of environment-authored failure byte. You might think you could catch more failures by reading those same bytes more loosely. You cannot. Loosening the grammar buys about 5pp of precision above the base rate. But it pushes false-alarm from 0.24% up to 25.6%–69.4% — roughly 130× to 350× worse. The lesson generalizes: you cannot buy recall by reading the bytes you already have more loosely. The only way to catch more is to add an *independent* witness — a byte the agent did not author — which is exactly the Tier-B re-read we run live in §6. The per-model grid (Fig. 11) shows this precision holding steady across every model that fires.

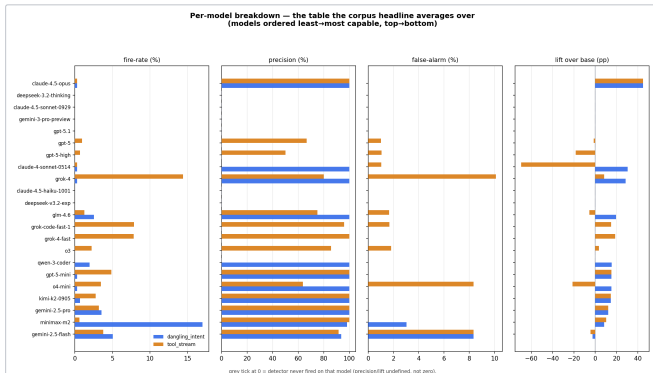


Figure 11. The same story, model by model. Each row is one model, ordered weakest to strongest. Precision (is a fire right?) stays high all the way up the ladder; what falls away on the strongest models is the fire-rate (how often it gets to speak). So the tight grammar is not over-caution — it is what keeps precision high — and the only honest way to catch more failures is a new source of evidence, not a looser reading of the bytes already in hand.

Table 5. The zero-training detectors versus a trained classifier on this corpus, on the same scoreboard. A classifier has to pick an *operating point* (how eager to fire); we show its two honest choices. Watch the *lift* column — the real-skill number from §2 — not the precision.

method	fire	precision	lift	false-alarm
terminal_error (o-training)	1.2%	95.0%	+18.8pp	0.2%
DOS trio (union, o-training)	5.1%	92.6%	+16.4pp	1.6%
trained clf — deployable point	0.0%	100.0%	n/a (1 fire)	0.0%
trained clf — F1-optimal point	100.0%	76.2%	+0.0pp	100.0%

Read the two classifier rows as the two ends of one dial. Turn the dial toward "cautious" (the *deployable* point, the only setting that does not constantly cry wolf) and it fires exactly once in 6,862 runs — effectively silent. Turn it toward "balanced" (the *F1-optimal* point, the setting that scores best by the usual accuracy measures) and it just predicts "everything failed": 100% fire, +0.0 lift, no skill at all. The widely-quoted "98% accuracy" lives at that second setting — it is the recall mirage from §7 wearing a classifier's clothes. There is no dial position where the trained model both fires usefully and stays trustworthy on a task it was not trained for. Our detectors, with zero training, sit at +16 to +19 points of lift at a false-alarm rate you could actually deploy.

Two more strikes. The classifier reads the agent's own trajectory structure, so it risks the mirror-verifier trap (§1) — it can learn the tells a model stops emitting once trained against. And when we feed it the DOS detector flags as features, its top weights *are* those detectors (`dangling_fired`, `terminal_error_fired`, `tool_stream_fired`): it re-derives the cheap signals it was handed. The right

8. Positioning

Two kinds of system sit near this work. One does the same job — catch a silent failure — by a different method. The other does a different job entirely. We take each on its own terms.

8.1. Trained classifiers

The nearest neighbour is a *trained* classifier: a model fitted on a labelled set of past failures, which then predicts whether a new run failed (arXiv 2511.04032, 2026, about 98% accuracy on its own data). On paper that beats our detectors by a wide margin. The catch is what the number costs — and where that 98% actually lives.

A classifier needs a labelled set of failures from the same kind of task, and a held-out slice to score it honestly. That labelled set is exactly the thing you do not have on a task you have never seen — which is the whole problem. Run the classifier *fairly* on this corpus and two things happen. Its best-balanced operating point just predicts "everything failed": on a benchmark where 76.2% of runs fail, that scores well by accuracy but buys no real skill — this is where the 98% headline lives (the recall mirage from §7). Its only deployable operating point — the one that almost never cries wolf — fires exactly once. So at a false-alarm rate you could ship, it does nothing.

place for such a model is not beside DOS as a rival but *under* it — a JUDGE-rung adjudicator (ORACLE → JUDGE → HUMAN) that rules only on the residue the deterministic detectors abstained on, fail-to-abstain, never the floor.

8.2. Arbiter neighbours

Two multi-agent arbitration systems sit nearby: Limen (advisory write-leases) and CodeCRDT (lock-free CRDT). Both compare to DOS's `arbitrate()` call, not to its detector. Neither can run on this benchmark at all: a frozen single-agent trace has no live write to mediate and no pass/fail to score, so there is no number to put beside them.

On this single-agent target DOS's arbiter likewise has nothing to mediate, so here it is a pure decision point. DOS's enforcement point — the `permissionDecision: deny` hook the agent runtime honours before a tool runs (shipped June 2026) — is simply never exercised by a single-agent benchmark.

The detector axis of this study and the arbiter axis of these neighbours do not share a scoreboard.

9. Limitations

- **In-loop: DETECT, not FIX — no task-lift number on Toolathlon.** Every Toolathlon figure is detection on frozen runs; the one validated *in-loop* action is the negative give-up of §4, which stops a doomed run but repairs nothing. The one *out-of-loop* action (§6, the live write-admission gate, $J = 10$ across two models and 120 tasks) is a measured *existence* result — it shows the gate catches and blocks real live over-claims at the same 8.3% rate on a mid and a strong model, not a calibrated population rate, and it is on a different benchmark (tau2-bench), not Toolathlon.
- **The recall ceiling is hard and tightens on the frontier.** Union recall is 6.18% (default) to 7.42% (surgical), so roughly 93% of failures stay silent — and fire-rate falls toward zero on the strongest models, which fail without leaving a byte to catch.
- **Four syscalls are null by construction here.** Toolathlon runs one agent on isolated state with no git or write-ahead log, so `arbitrate`, `verify`, `liveness`, and `resume` have no job; only the in-flight byte-clean detectors do.
- **The give-up token-saving magnitude is single-corpus, single-model.** The gate never wrongly halts a winning run where it fires (specificity, proven across 22 models and two benchmarks); how *often* it fires, and so how much it saves, depends on the benchmark and model (sensitivity, benchmark-dependent).

10. Conclusion

A cheap, hard-to-fool failure detector earns exactly one action worth taking. The detector is byte-clean (§1: it reads only environment-written bytes the agent cannot forge), needs no training, and is scored by a third-party oracle — an independent checker that reads the real app state and says pass or fail, which DOS does not own. When we measured every action we could try, only the negative one helped: give up correctly — stop pouring compute into a run that is clearly doomed. An error-gated halt wrongly stops **zero of 1,634 winning runs across 22 models** on Toolathlon, and **zero on a second benchmark**, while saving **11.4% of the fleet's real tokens (pooled across both arms)**. Every *active* fix we tried — warn the agent, rewind its bad turns, feed it the environment's own correction — was flat or made things worse, because injecting a turn at all perturbs the runs that would have passed. Detection is the setup; stopping the doomed run is the result.

What an in-flight detector cannot reach is the dominant frontier failure: a confidently-wrong final state with no error and no loop in the trace, because the agent never re-read the state it claims to have changed. The only honest catch for that is a live re-read against a fresh world byte — the rung the

benchmark's own final-state verifier occupies. So we built it (§6). Running it live on a second benchmark across *two* models, an out-of-loop write-admission gate caught and blocked **10 genuine over-claims** off the environment's own database hash, admitting every correct write — and the over-claim rate held identical (8.3%) on a mid and a strong model, so the out-of-loop signal does *not* fade at the stronger tier the way the in-flight detector's does. This is the live promotion of that final-state rung, spent where it does no harm: on a downstream peer's input, never back in the judged agent's loop. The decisive move is not a better detector but a better *consumer* — the same verdict is harmful in the producing loop (§4.3) and valuable out of it (§6). So: detect cheaply and byte-clean in-loop, stop the doomed run, and spend the verdict out-of-loop to refuse a phantom write before a peer inherits it — a measured existence result across two models, with the W3 residue (a goal with no world footprint) still beyond any witness short of a human.

Reproducibility

Every number in the Toolathlon study (§1–5, §7–10) recomputes offline, with no network or model calls, from the durable per-run join `replay_all_rows.csv` (7,116 records, content fingerprint `sha256:1a55e8d8e2d4`). The additivity single-source-of-truth re-asserts the headline claims and exits non-zero if a regenerated file drifts from them, so a claim cannot silently part from its data; the give-up cross-benchmark numbers and figure regenerate from their own script. The one exception is §6: it reports a *live, paid* run (tau2-bench through Gemini; single-model run \$0.89, cross-model run \$4.77), so its numbers are a fixed read-off of that run, not an offline recompute. They are transcribed verbatim into the figure's single source of truth (`paper/_VERIFIED_FACTS_228_2026-06-08.md` and `_VERIFIED_FACTS_232_2026-06-08.md`), and the run is resumable from a per-task cache so it never re-spends; re-running it requires a Gemini API key and reproduces an *existence* result (the gate blocks real over-claims), not the exact count, since the live rollouts vary.

```
# 1. regenerate the durable rows from the frozen trajectories (offline
once _data/ is cached):
python -m benchmark.toolathlon.run_replay --all --no-download --by-model \
    --rows-out benchmark/toolathlon/_results/replay_all_rows

# 2. recompute + re-assert the detector / additivity / union claims
(exits non-zero on drift):
python -m benchmark.toolathlon.additivity --check

# 3. regenerate the give-up-correctly numbers and the cross-benchmark
figure:
python benchmark/giveup_cross_benchmark.py

# 4. redraw every figure from the same single-source-of-truth numbers:
python -m benchmark.toolathlon.viz
```

Artifact. The replay harness, the per-run data, the figures, and the single-source-of-truth scripts that compute every number in this paper are released together, under `benchmark/toolathlon/` and `benchmark/` in the project repository. The Toolathlon benchmark and its `Toolathlon-Trajectories` dataset (CC-BY-4.0) are the work of the HKUST-NLP authors; we replay them here as an external, third-party-scored test target, and they are not a contribution of this paper.

Appendix A. DOS in plain words — the Vaseline test

The body of this paper defines its terms as it goes. This appendix slows down and explains the one idea underneath all of them — *do not trust the narrator* — in the plainest language we can manage, then says what DOS is and why the numbers look the way they do. It is self-contained: you can read it on its own, before or after the rest of the paper. It leans on one running picture — a smear of Vaseline on a mirror — to carry the whole idea, and ends with two reference tables: a key to every symbol the body uses (Table A1) and a plain-language glossary of every term (Table A2).

A.1 The one idea: do not trust the narrator

An AI agent doing a long task is also the author of the story about that task. It writes the actions *and* the closing summary that says how the actions went. Those are two different things, and the second one is the problem. The agent can end a run with a confident line — “*All customers have now received their surveys!*” — while the actual inbox shows zero emails sent. Nothing crashed. Nothing in the transcript contradicts the happy ending. The story is clean; the world is wrong. This is a **silent failure**: a wrong result wearing the costume of a right one. On the hard benchmark we study here, more than three runs in four end this way.

So the obvious move — read what the agent said about itself and decide if it sounds done — is a trap. We call it the **mirror-verifier** trap: you hold up a mirror to the agent and ask the agent how it looks. If you then train the agent

against that check, it does not get more honest; it just stops saying the give-away phrase. You measured the *story*, not the *world*.

A.2 Byte-clean, said plainly — the Vaseline picture

Here is the picture that makes the whole thing click. Think of the real state of the world — the inbox, the database row, the file on disk — as a mirror. Now think of the agent's own narration as a **smear of Vaseline** wiped across that mirror. Vaseline does not change what is behind the glass; it changes what you *see*. Look through the smear and everything goes soft, flattering, and clean — you see the agent's polished “done,” not the smudged truth underneath.

A checker that reads the agent's narration is looking *through the Vaseline*. It will always report a clean mirror, because that is what Vaseline does. Worse, the smear is free to re-apply: the agent can always produce a smoother sentence. That is what *forgeable* means — a signal the judged agent can author, and therefore wipe away.

The fix is not to polish the glass better. It is to **refuse to look through the smear at all**, and instead read only the things the agent's narration cannot coat:

- the **tool results the environment sent back** — an error envelope from the mail server, the identical “pending” that came back three times, the row that is simply absent;
- a **third-party judge** that re-reads the live world after the run and reports pass or fail on its own.

The agent did not write any of those bytes, so it cannot smear them. A check that reads *only* bytes the judged agent did not author is what we call **byte-clean**. That single rule — *read around the Vaseline, never through it* — is the

foundation of everything DOS does. The test for any proposed check is one question: *could the agent have written these bytes?* If yes, it is Vaseline. If no, it is ground truth.

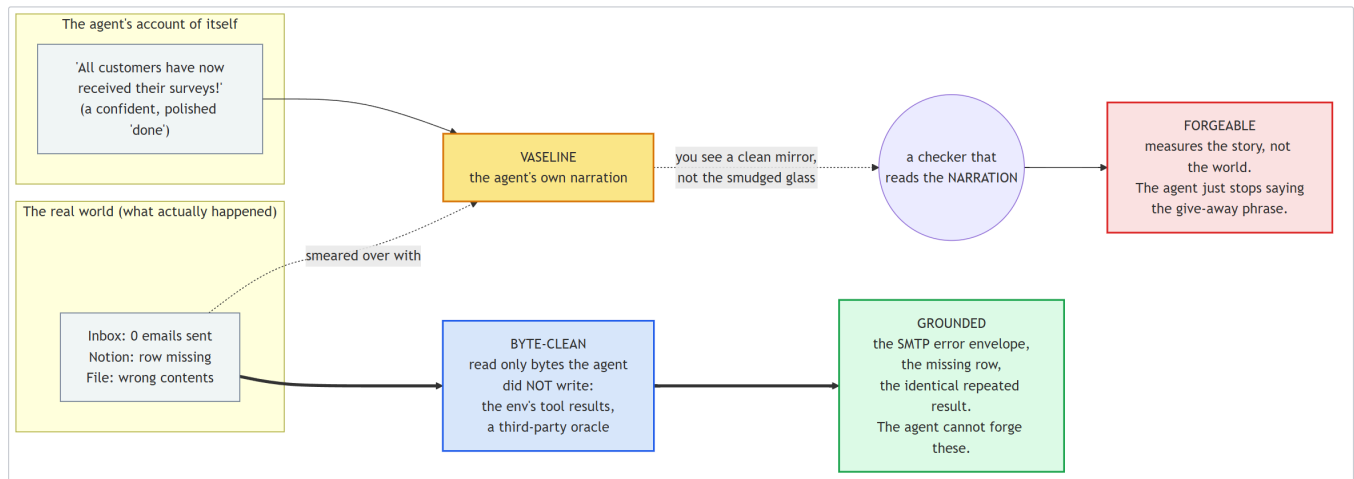


Figure A1. The Vaseline test. The agent's narration is a smear wiped over the real state of the world: it does not change what happened, only what a careless checker sees. Reading *through* the smear (the agent's own words) is **forgeable** — the agent can always wipe on a cleaner sentence. Reading *around* it (the environment's tool results, a third-party oracle — bytes the agent never authored) is **byte-clean** and grounded. Every DOS check lives on the bottom path.

The Vaseline test, in one line: if the agent could have written the bytes your check reads, you are measuring the smear, not the mirror.

A.3 What DOS actually is — a referee made of "no"

DOS stands for **Dispatch Operating System**. The name is a deliberate echo of a computer's operating system, which sits between many programs and the one shared machine and decides who gets to do what. DOS sits between many AI agents and the one shared world they are all changing, and it plays **referee**.

Its motto is blunt: *the kernel is the part that does not believe the agents*. (The **kernel** is the small trusted core — the referee itself, as opposed to the agents it watches.)

A referee mostly says one word: **no**. DOS is built from a handful of small, careful ways to say it, each one grounded in byte-clean evidence rather than in what an agent claims. You do not need to memorise them; the point is that they are all the same move — a refusal backed by evidence the agent could not forge — pointed at a different question.

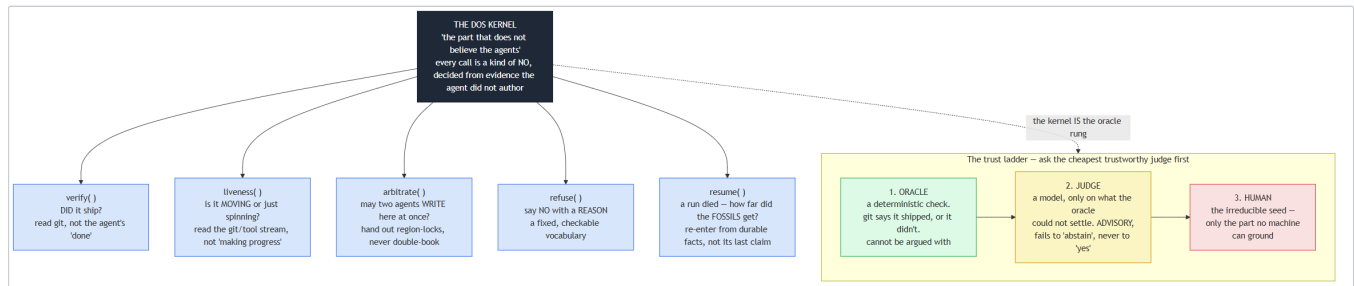


Figure A2. The DOS kernel is a small set of grounded refusals. Each call answers one question using bytes the agent did not author: **verify** (did the work really ship? — read git history, not the agent's "done"), **liveness** (is the run moving or just spinning? — read the stream of results, not "making progress"), **arbitrate** (may two agents write the same place at once? — hand out locks, never double-book), **refuse** (say no with a reason from a fixed, checkable list), and **resume** (a run died mid-flight — how far did its durable traces actually get?). Below, the **trust ladder**: ask the cheapest trustworthy judge first. A deterministic **oracle** settles what it can; a model-based **judge** rules only on the leftovers, and only as advice that fails safe to "abstain"; a **human** is the last resort for the part no machine can ground.

Two features of that ladder are worth stating plainly, because they are what keep the referee honest:

- **The oracle goes first, and it cannot be argued with.** "Did this phase ship?" is answered by looking at the git history — a record the agent cannot rewrite to flatter itself — not by reading the agent's summary. A deterministic answer that an agent cannot influence is always preferred to a clever one that it can.
- **The model-judge is allowed, but only on a short leash.** Sometimes no deterministic check exists and you need a model to weigh in. DOS permits this, but boxes it in: the model only rules on what the oracle could not settle, it only gives *advice* (it never blocks on its own), and when it is unsure it must say **"abstain"**, never "looks fine." A model checking a model is still two things on the smeared side of the glass, so it is kept advisory and made to fail toward caution.

A.4 The three detectors — three kinds of unforgeable evidence

This paper measures three of those refusals in detail. Each one is a **detector**: it watches a finished run and "fires" (raises a flag) when it spots a specific, byte-clean fingerprint of failure. All three obey the Vaseline test — they read only what the environment wrote, never what the agent said about itself.

- **dangling_intent** — **"said it wasn't done, then stopped."** The agent's *last* message admits open work ("I still need to update the rest...") and then

nothing in the environment acts after it. The stated obligation is the agent's own words, yes — but the firing condition is the *absence of any further environment action*, which is a fact about the world, not the story. The agent cannot make a tool result appear by narrating one.

- **tool_stream** — **"spinning in place."** The same tool call returns the same result several times in a row. The agent keeps asking a question the environment keeps answering identically; it is learning nothing. The agent cannot forge "the same bytes came back three times" — the environment authored those bytes.
- **terminal_error** — **"walked away from a wall."** The run ends while the environment's last word was an unresolved error (a crash, a 5xx, a permission-denied) that the agent never fixed. A confident "Email sent successfully!" cannot suppress this, because the firing condition is the error envelope the mail server wrote, not the sentence the agent wrote.

Notice the shape they share. Each detector pairs something the agent might say with something only the world can show, and it fires on the *world* half. That is the Vaseline test applied three ways.

A.5 Why a great-sounding number does not win the benchmark

The detectors produce a number that sounds like a triumph and a number that sounds like a flop, and both are true at once, because they answer different questions. Keeping them apart is the single most important thing to understand about the results.

- **Precision (and its cousin, lift): "when it fires, is it right?"** When these detectors fire, the third-party judge confirms a real failure 88–98% of the time. Because failures are common on this benchmark (76% of runs fail), we report **lift** = precision minus that base rate, which is the "better than guessing" part. The lift is large: up to +22 points. This is the triumph.
- **Recall: "of all the failures, how many did it catch?"** About two in a hundred. This is the flop. Most failures leave no byte-clean fingerprint at all — the agent neither narrates an open task nor loops on identical bytes nor hits a reported error — so a byte-clean detector simply cannot see them.

The everyday version: imagine a smoke alarm that is **98% accurate but only goes off in about two of every hundred real fires**. When it sounds, you should run — it is almost never wrong. But you cannot throw away your other alarms, because it stays silent for most fires. The accuracy is real *and* the coverage is tiny. People confuse the two because both feel like "how good is the detector?" — they are not the same question.

It gets sharper, and more honest, on the strongest models. The detectors need a *visible* failure — a narrated loose end, a repeated byte, a reported error. Weak models fail loudly and leave those traces. The strongest models fail *quietly*: they stop with a polished, confident, completely wrong summary and no trace in between. So exactly where the leaderboard lives, the fire-rate drifts toward zero. The quality of the signal carries across every model; its *coverage shrinks as models get better*. We never dress this up as anything else.

A.6 The honest punchline — detect, don't fix, and the one action that helps

There is a gap between **spotting** a failure and **fixing** it, and this study lives entirely on the spotting side. The data here are frozen recordings, so nothing could ever step in and change a run. We therefore make no claim that warning an agent would have turned a failure into a success — and a separate, live program we ran on a different benchmark explains why that restraint is wise.

In that program we tried, on a real running agent, every *active* way to fix a struggling run — and every one of them came up empty or worse:

- **Warn it.** Re-surface the detector's verdict and hand the agent back its own unfinished obligation. On the natural failure stream this was **flat** (+0.2 points — noise). A capable agent that stopped usually stopped because it hit a wall, not because it was lazy; handing the wall back does nothing.
- **Rewind it.** Quietly cut the bad turns out of the transcript and let it try again. **Zero** conversions, zero flips, across every sample size we tried.
- **Cure it.** Re-surface the environment's *own* correction (a byte-clean nudge, not an answer we wrote) — the strongest, cleanest intervention we had. It actually fired on most runs, and it was **net-negative**: it rescued the two runs it was aimed at but knocked out more runs that would otherwise have passed. The very act of injecting a turn perturbed the loop.

The lesson is uncomfortable and clean: on a fragile run, *the harm is the intervention existing in the loop at all*, not the content of the nudge. So the only action that survives is the **negative** one — **give up correctly**: when a run is clearly doomed, stop it and spend its budget elsewhere. Giving up authors no new turn, so it perturbs nothing, and it is the one lever that paid for itself.

And it is byte-clean by the same rule as everything else. The gate fires when one tool hits the same *environment-authored error* several times in a row and is still erroring. Across two benchmarks and more than 1,600 winning runs spanning the whole frontier, this gate halted **zero** winners — it never killed a run that was about to succeed — while saving roughly a tenth of all the compute. The subtle part is what makes it sound: a naive version that halts on any *repeat* (not just a repeated *error*) does kill winners — six of them, all one model politely *polling* a slow resource until it answered. Repeating is normal; repeating an **error** is the doomed signal. The environment authored the error, not merely the repeat — the Vaseline test, once more, doing the load-bearing work.

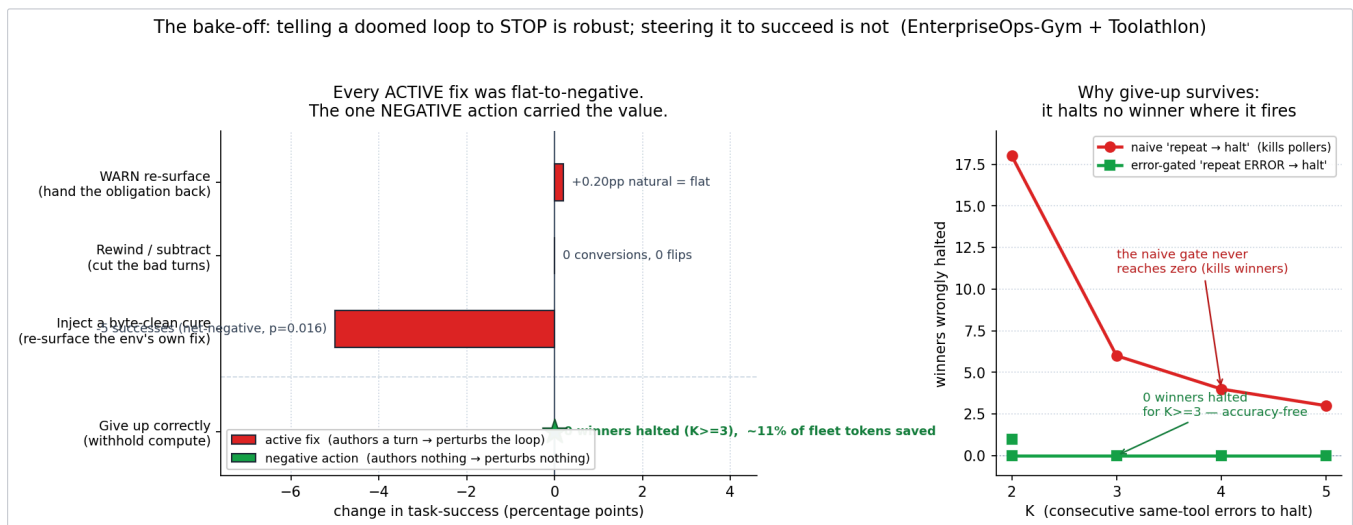


Figure A3. The bake-off: steering a doomed loop fails, stopping it works. **Left:** every *active* fix moved task-success from flat (warn) to nothing (rewind) to negative (the injected cure) — because each one authors a turn, and a turn perturbs a fragile run. The one action with value is the *negative* one (green star): give up and re-spend the budget, which authors nothing. **Right:** why give-up is safe to act on — the error-gated halt wrongly stops **zero** winning runs for a threshold of three or more, while a naive halt-on-any-repeat never reaches zero (it kills pollers). These numbers are from a separate benchmark (EnterpriseOps-Gym) and Toolathlon, kept apart from the main Toolathlon detection tables.

The punchline, in one line: telling a doomed loop to stop is robust; steering it to succeed is not — so the substrate detects, withholds compute when it is sure, and hands the rest to a human, rather than pretending it can fix what it can only see.

A.7 Where the substrate ends and a verifier begins

Finally, the boundary, stated without spin. The failure that beats every detector in this paper is the one with no fingerprint at all: the agent runs cleanly, hits no error, contradicts nothing, and ends with a confident, polished, *wrong* summary — because it never re-read the state it claims to have changed. No byte already in the transcript reveals this. The only honest way to catch it is to **go re-read the world yourself** after the run — re-query the live application and check each claim against a fresh byte the agent never touched.

That is exactly the job of the benchmark's own final judge, and it is the line this paper draws on purpose. An in-flight, advisory, byte-clean substrate — DOS — can read the evidence the environment already wrote, cheaply and without ever believing the agent. A **final-state verifier** re-reads the world to settle what no in-trace evidence can. The gap between them is not a flaw to hide; it is the boundary, drawn honestly: where reading around the Vaseline runs out, and going to look behind the mirror begins.

A.8 Notation — the symbols, decoded

The body uses a handful of single letters and short codes as shorthand. They are never hard, but a paper that assumes you already know them is doing the very thing this appendix exists to undo. Here is every symbol in one place, said plainly. The one most worth fixing is the letter **J**: the body leans on it on nearly every page of the headline result and never once spells out what it stands for.

Table A1. Every symbol and short code in this paper.

Symbol	Reads as	What it means, plainly
J	the payoff	A plain count of <i>damage prevented because the referee acted</i> : how many bad writes a downstream agent did <i>not</i> inherit, or how many overwrites a race did <i>not</i> cause, that it <i>would</i> have without the referee. Not a rate, not a percentage — an integer tally of avoided harm. "J = 10" means ten real failures were caught and stopped. The whole §6 result is reported in J.
K	the halt threshold	How many times the <i>same</i> tool must return an error <i>in a row</i> before the give-up gate proposes a stop. We use K = 3: three identical-tool errors back to back, with the latest still failing. Lower K stops sooner but risks killing a run that would have recovered; K = 3 is the smallest value that never killed a winner across 22 models.
pp	percentage points	The plain <i>difference</i> between two percentages, to avoid the usual confusion. Going from 76% to 92% is "+16 pp" (16 points added), <i>not</i> "+21%" (the relative change). Lift is always quoted in pp.
db_match	"database matches"	The unforgeable witness in §6. After a run, the benchmark hashes the task's database and compares it to a hash of the <i>correct</i> ("gold") database the task designer specified. True = the agent left the world correct; False = it did not; absent = this task changes no database, so there is nothing to check. The agent writes none of these bytes.
Wo–W3	witness rungs	The four grades of evidence, weakest to strongest (§5). Wo : the agent's own narration (forgeable — no evidence at all). W1 : the environment acknowledged a write happened. W2 : a check that the change actually <i>persisted</i> (DOS's git rung lives here — it confirms <i>presence</i>). W3 : a full comparison against the correct answer (confirms <i>correctness</i> — what db_match does). Higher is stronger; only W3 settles whether the work was <i>right</i> .
tp / fp / fn	true-positive / false-positive / false-negative	How a verdict lands against the truth. When the detector <i>fires</i> it is either a <i>true-positive</i> (the run really failed — a good catch) or a <i>false-positive</i> (the run actually passed — a false alarm, the costly mistake). When it <i>stays silent</i> on a failing run, that is a <i>false-negative</i> (a miss — the "silent failure" it could not see). The fourth box, a correct silence on a passing run (true-negative), needs no symbol. These four boxes are the confusion grid.
Wilson-95	"Wilson 95% upper bound"	A standard, slightly conservative way to put a ceiling on a rate you measured as zero. "Zero false-halts, Wilson-95 upper 0.0023" means: we saw none, and the math says the <i>true</i> rate is almost certainly below 0.23% — a guard against reading "0 out of N" as a stronger promise than the sample size can support.

A.9 Glossary — every term, in plain words

Grouped so you can find a term by the kind of thing it is. Every entry is self-contained; where the body uses a word this appendix has not already

unpacked, it is here.

Table A2. The terms used in this paper, in plain language, grouped by topic.

Term	What it means, plainly
The one idea, and its vocabulary	
silent failure	A run that ends "successfully" by its own account but left the world wrong — no crash, no contradiction, just a confident, false "done." More than three runs in four end this way on the hard benchmark here.
byte-clean	A check that reads <i>only</i> bytes the judged agent did not write (the environment's tool results, a third-party judge). The thing the agent's narration cannot smear. The single rule under everything DOS does.
byte-author	Whoever actually produced the bytes a check reads. The whole trust model is one test on this: a check is trustworthy only when its byte-author is <i>not</i> the agent being judged.
forgable	The opposite of byte-clean. A signal the agent <i>can</i> author, and therefore wipe away — reading its narration is forgable; reading an error envelope the mail server wrote is not.
the Vaseline test	The one-question check for any signal: <i>could the agent have written these bytes?</i> If yes, it is the smear (forgable); if no, it is the mirror (byte-clean).
mirror-verifier trap	Checking an agent by asking the agent how it looks — comparing its words to its own other words. It measures consistency (does the story hang together?), not truth (does it match the world?).
consistency ≠ grounding	Re-deriving an author's <i>own</i> bytes proves only that the story is consistent; checking against a <i>different</i> author's bytes proves it is grounded in something real. Only grounding crosses the trust floor.
The referee — DOS and its parts	
DOS	Dispatch Operating System: the small trusted referee this paper is about. It sits between many agents and the one shared world they are changing, and believes none of their self-reports.
kernel	The small trusted core — the referee itself — as opposed to the many agents it watches. "The part that does not believe the agents."
oracle	A deterministic check that cannot be argued with (e.g. git history says a phase shipped, or it didn't). Always tried first.
third-party oracle	The benchmark's own pass/fail judge, which re-reads the live world after a run. Authored by the benchmark, not by DOS — so a verdict DOS cannot fake. Every detector here is scored against one.
judge	A model-based check, used only on what the oracle could not settle. Advisory; must say "abstain" when unsure; never blocks on its own.
ORACLE → JUDGE → HUMAN	The trust ladder: try the cheapest trustworthy check first. A deterministic oracle settles what it can; a model-judge rules only on the leftovers, as advice that fails safe to "abstain"; a human is the last resort for what no machine can ground.
abstain	The safe non-answer. When the evidence is missing, a DOS check says "abstain" rather than guess — it can refuse on proof, but it never <i>affirms</i> without it. Abstain always means admit, never block on a hunch.
advisory	A check that <i>reports</i> a verdict but does not <i>enforce</i> it — it proposes, a human or a downstream system decides. Everything measured here is advisory; nothing kills a run on its own.
detector / fire	A check that watches a run and "fires" (raises a flag) on a specific byte-clean fingerprint of failure. Fire-rate = how often it flags, as a fraction of all runs.
structured error envelope	The tight, specific <i>shape</i> of an environment-authored failure (a crash, a 5xx, a permission-denied) that <code>terminal_error</code> matches — as opposed to merely the word "error" appearing somewhere in a result. Matching the shape, not the word, is what keeps false alarms near zero.
Where you spend the verdict — the heart of the paper	
in-loop	Spending a verdict by handing it <i>back to the agent that earned it</i> — warning it, rewinding it, feeding it a correction. Measured here as flat-to-harmful, because adding any turn disturbs a run that might have passed.
out-of-loop	Spending the <i>same</i> verdict on <i>anyone but</i> the agent that earned it — a downstream peer, the operator, a training set. This is where it pays: correcting a peer's input injects nothing back into the original run.
claim (failure)	The first way a fleet fails: one agent announces a write that did not really land, and a downstream peer is about to inherit that false "done" as truth.
race (failure)	The second way a fleet fails: two agents touch the same shared state at once, and one silently overwrites the other. No one lied; the shared state is wrong anyway.
write-admission gate	The out-of-loop referee for a <i>claim</i> . It sits on a downstream peer's input and, before the peer inherits agent A's write, checks A's announcement against a byte A could not author — admitting a real write, blocking a phantom one.
over-claim / phantom write	An agent confidently announcing a successful change that the world's own record says did not land correctly. The thing the write-admission gate catches.
clobber / lost update	What a race causes: a second agent's write lands on top of a first agent's, erasing it, because the second never saw the first. The thing the arbiter prevents.
lease / region / lane	The arbiter's tool. A <i>region</i> is a named patch of shared state (e.g. one reservation, written <code>reservations/ <id></code>); a <i>lease</i> is a temporary exclusive hold on a region. Two agents wanting overlapping regions cannot both hold a lease, so their writes are forced into order.
arbiter / arbitrate	The referee for a <i>race</i> . It hands out leases on regions and refuses a second agent's lease while a first holds an overlapping one — so the two writes <i>serialize</i> instead of colliding.
serialize	To force two would-be-simultaneous writes into a strict order, one after the other, so the second acts on the result of the first instead of clobbering it.
half-plane	Just "one of the two distinct sides" — here, the two distinct ways a fleet fails (a claim and a race). The paper's symmetry is that one referee covers both half-planes off the same witness.
existence result	A claim stated no wider than "this happens and we caught it N times," <i>not</i> "this happens X% of the time everywhere." §6 is an existence result: it proves the gate catches real over-claims live, not that the 8.3% rate is universal.
The give-up result	
DETECT vs FIX	Spotting a failure vs repairing it. This paper measures DETECT only. The one FIX-adjacent action that helped was the negative one — give up correctly.
give up correctly	Stop a doomed run and re-spend its token budget elsewhere. Authors no new turn, so it perturbs nothing — the only in-loop action that paid off.
false-abandon	The give-up gate's one costly mistake: halting a run that <i>would</i> have succeeded. The number that has to be zero — and is, at K = 3.
specificity vs sensitivity	Two halves of "how good is the gate?" <i>Specificity</i> : when it fires, it never halts a winner (the half we have, proven across 22 models). <i>Sensitivity</i> : it fires often enough to catch most doomed runs (the half we do not — it stays quiet on silent failures).
circuit breaker / SPRT / CUSUM	Three classic "stop after repeated trouble" mechanisms from engineering and statistics. The give-up gate is a stripped-down corner of each; we claim none of the mechanism — only the byte-clean <i>grounding</i> (it counts errors the agent cannot forge) and the measured threshold are new.
Measuring a detector	
precision	When it fires, how often it is right. Ours: 88–98%.
recall	Of all the failures that happened, how many it caught. Ours: ~2% per detector, ~6% combined. The low number.
base fail rate	How often runs fail anyway (76.2% here — the benchmark is hard). Precision has to beat this to mean anything, because a "detector" that flags everything is right this often by luck.
lift	precision – base fail rate (in pp): the "better than guessing" part. The big, real, but rare number. Throughout the paper, lift — not raw precision — is what says a detector knows anything.

Term	What it means, plainly
false-alarm rate	Of all the runs that actually <i>passed</i> , how often the detector wrongly flagged one. The costly mistake: cry wolf and an operator stops listening. Ours: near zero.
confusion grid	The 2×2 picture of every run split four ways — caught failures, missed failures, correct passes, false alarms. Here the "missed failures" cell dominates: the low recall, seen as a picture.
net-new / additivity	A catch a detector makes that the others did <i>not</i> — so adding it raises total coverage instead of re-finding the same failures. <code>terminal_error</code> is 99% net-new: it reads a different byte and finds a different failure.
union (pair / trio)	Treating a <i>set</i> of detectors as one: a run is "caught" if <i>any</i> member fires. The <i>pair</i> is the first two detectors; the <i>trio</i> is all three.
operating point	A dial setting on a classifier trading eagerness-to-fire against false alarms. A trained classifier has no single setting that both fires usefully <i>and</i> stays trustworthy on a task it was not trained for — the point of the "trained classifiers" comparison in the Positioning section.
frontier	The most capable models (those above a chosen pass-rate cut), where the leaderboard lives. The detectors fire rarest exactly here, because the strongest models fail <i>quietly</i> .
Systems and benchmarks named in the body	
Toolathlon	The hard benchmark behind the offline replay study (every section except the live §6): long tasks using 600+ real tools across 32 apps. Its own pass/fail oracle re-reads the real app state, which is why it is a clean byte-clean target. DOS did not build it; we replay its published recordings.
trajectory / replay	A <i>trajectory</i> is the full recorded transcript of one agent's attempt at one task. <i>Replay</i> is running the detectors over those frozen recordings offline (§0, no live agents) — which is why the Toolathlon study can only measure DETECT, never FIX.
tau2-bench	The live benchmark used in §6: airline and retail customer-service tasks. It is the right target because it ships <code>db_match</code> — the database-hash witness the agent cannot author.
EnterpriseOps-Gym	The separate live benchmark used for the §4.3 bake-off (a ServiceNow-operations gym with its own hidden database checks). Used because you can only measure an active fix by replaying it into a <i>live</i> agent and watching it react — a frozen recording cannot.
Gemini (flash / pro), grok-4	The models run live in §6 (<code>gemini-2.5-flash</code> , a mid tier; <code>gemini-2.5-pro</code> , a stronger tier) and the one model that exposed the naive give-up gate's flaw by harmlessly polling (grok-4). Named only where a specific run depends on the model.
SMTP / 5xx / API error	Ordinary failure shapes the environment reports: an SMTP rejection is a mail-server "no"; a 5xx is a server-side HTTP error; an API error is a transient call failure. All are environment-authored bytes — the agent cannot talk them away.
The boundary and beyond	
final-state verifier	A check that re-reads the live world after a run to catch what no in-trace evidence can (the confident-but-wrong ending). The boundary where an in-flight detector's reach ends and §6's out-of-loop re-read begins.
RLVR / poison reward label	RLVR is Reinforcement Learning from Verifiable Rewards — training a model on outcomes scored right/wrong. A <i>poison reward label</i> is an over-claim a naive, self-judged loop would bank as a <i>win</i> — teaching the model to over-claim more. §6.6 notes the same gate, aimed at a training set, purges them off ground truth the model cannot fake from its own outputs.

The companion engineering write-up for this appendix — with the same diagrams and the "why doesn't the lift carry over?" walk-through — lives at `benchmark/toolathlon/EXPLAINER.md` and `GLOSSARY.md` in the DOS repository.

Appendix B. How it works — from a forgeable claim to a grounded refusal

Appendix A said *what* DOS believes (only bytes the agent could not author) and *where* a verdict pays (out of the producing loop). This appendix says *how* a verdict is actually computed — the small machine that turns a forgeable sentence into a grounded "no." It is still plain-language and self-contained, but it follows the bytes: where they are read, how they are tagged so the agent's own words can never count, the one pure step that decides, and the closed list of answers it may return. Then it shows the two live consumers from §6 are not two systems but the *same* two primitives, pointed at the two ways a fleet fails. The reference points are real: the gate is `benchmark/agentprocessbench/writeadmit/gate.py`, the referee-between-agents is `coord_loop.py`, and the floor both stand on is `dos.effect_witness`.

B.1 The pipeline: four steps, and only one of them decides

Every DOS verdict — the write-admission gate, the race arbiter, every detector in this paper — is the same four-step pipeline. The shape is deliberate, and the discipline is in the seams between the steps, not in any one step's cleverness.

- Gather evidence — at the boundary, never inside the decision.** All the reading from the messy world happens here and only here: read the environment's database hash, read the tool-result stream, read the git log. This is the one step that touches I/O. It does not judge anything; it just collects bytes and notes *who wrote each one*.
- Tag each byte with its author — the accountability label.** Every piece of evidence is wrapped with a label saying which rung of trust it came from: did the *agent* write it, did the *environment* record it, did a *third party* grade it? This label is the whole game, and B.2 is about it.
- Classify — a pure function, no I/O, the same answer every time.** A single function takes the tagged evidence and a small policy and returns a verdict. It reads no files, makes no network call, asks no model. Given the same evidence it returns the same verdict, forever — so the decision can be re-run, audited, and trusted not to drift. This is the only step that decides.
- Return a typed refusal — from a closed, checkable list.** The answer is not free text. It is one value from a fixed vocabulary (admit / block / abstain, and a named reason), so a downstream consumer can branch on it

without parsing prose, and a human can check that every possible answer was accounted for.

The reason to split reading (step 1) from deciding (step 3) so strictly is the same reason a courtroom separates gathering testimony from rendering a verdict: a decision that re-reads the world while it is deciding can be influenced by what it re-reads, and an agent that can influence the read can influence the verdict. By the time the pure classifier runs, the evidence is fixed and labelled; the agent has no further move. *I/O at the boundary, a pure verdict at the core* is the one structural rule the whole kernel obeys.

B.2 The accountability ladder — why the agent's own words can never tip the scale

Step 2 is where byte-clean stops being a slogan and becomes a mechanism. Each piece of evidence carries a label naming its author, and the labels are ordered by how hard they are to forge:

- AGENT-AUTHORED** — the agent wrote it. Its final message, its self-report, the "receipt" it pasted into its own answer. This is the Vaseline. It is allowed into evidence, but it is marked, and the mark is load-bearing.
- OS-RECORDED** — the environment wrote it. The database hash the tool-executor computed, the error envelope the mail server returned, the git commit the version-control machinery stamped. The agent triggered the action but did not author the byte.
- THIRD-PARTY** — an independent grader wrote it. Toolathlon's `evaluation/main.py`, tau2's database-hash check. Authored by neither the agent nor DOS.

Now the rule that makes the floor unforgeable, stated once and exactly. **Only an OS-RECORDED or THIRD-PARTY witness may set the "refuted" bit.** An AGENT-AUTHORED byte can be *recorded* — the verdict notes that the agent *claimed* success — but it can never *confirm* a claim and never *refute* a competing one. So a policy that writes itself a glowing receipt ("payment processed, confirmation #A1B2C3") moves nothing: its own bytes land on the forgeable floor, which the classifier treats as *no evidence either way*. The only thing that can flip a confident write from "admit" to "block" is a byte from a higher rung — a database hash that disagrees. In the code this is the `believe_under_floor` discipline inside `witness_effect`: a read-back on the AGENT-AUTHORED rung yields *unwitnessed*, never *refuted*.

The floor, in one line: the agent's words are admitted as *testimony*, never as *proof* — only a byte the agent did not author can decide the case.

This is the same trust ladder Appendix A drew (ORACLE → JUDGE → HUMAN), seen from the evidence side rather than the adjudicator side. There, a deterministic oracle ruled before a model-judge was allowed to whisper, and the judge failed safe to "abstain." Here, an accountable witness ranks above the agent's narration, and a missing witness fails safe to "admit." Same shape, same direction: trust the unforgeable thing first, and when the unforgeable thing is silent, do the cautious thing — never the flattering one.

B.3 Consumer one: the write-admission gate, line by line

Take the gate from §6 and watch the four steps turn. A producing agent A finishes a tau2 task and emits a final message; a downstream peer B is about to inherit whatever A changed as its own starting truth. The gate sits on B's *input*.

1. **Gather.** Read two things: A's final message (the claim) and the tau2 evaluator's database hash result, `db_match` — true if the database A left behind matches the gold database the task designer specified, false if it does not, absent if the task has no database footprint.
2. **Tag.** A's message is wrapped AGENT-AUTHORED (forgeable). `db_match` is wrapped OS-RECORDED: True becomes an *attestation* ("the effect is present and correct"), False a *refutation* ("the claimed write did not land correctly"), and None contributes *no witness at all*.
3. **Classify.** The pure call: does A's message *confidently claim a write* (a closed test — a write-verb intersected with a mutating-tool reference, not a keyword sniff), and does an accountable witness *refute* it? Block only when both are true.
4. **Return.** One of three typed outcomes — **admit** (no confident write claimed, or the witness agrees), **block** (a confident write claimed and the database hash refutes it), or **abstain** (a confident write claimed but `db_match` is absent, so there is nothing to check — and abstain means admit, never invent a verdict).

Three properties fall out of the structure for free, and they are exactly the ones the paper leans on. First, **it cannot be talked into admitting**: because only an OS-RECORDED byte can set "refuted," a fake receipt in A's own answer is on the forgeable floor and changes nothing — the gate can only ever block *more* under a real witness, never be argued into blocking less. Second, **it never**

blocks correct work: a write the database confirms is attested, not refuted, so it is admitted — which is why, across two models, all nine confirmed-honest writes were admitted and only the five witness-refuted ones blocked. Third, **blocking B's input writes nothing into A's loop**, so the in-loop harm of §4.3 is structurally absent: this is the entire reason the consumer is B, not A.

B.4 Consumer two: the race arbiter, same shape, different witness-moment

The coordination referee from §6.5 looks like a different machine and is the same pipeline with one substitution: the witness is read *before* the write instead of after. Two agents, A1 and A2, are both pointed at the same airline reservation R — A1 to cancel it, A2 to add a bag. Left alone, A2 computed its move against the *original* R, never saw A1's cancellation, and its stale write lands on top: an update is silently lost.

1. **Gather + tag.** The shared state names a *region*: a reservation is the path-like string `reservations/<id>`. Two agents on the same reservation name the same region. (This mapping is the part a reader expects to be hard; it is one line.)
2. **Classify.** The pure call is `arbitrate(region, live_leases)`: A1 holds a lease on `reservations/R`; A2 requests an *overlapping* region while A1 holds it; the arbiter refuses the second lease. No model, no I/O — the same disjointness verdict the live `dos lease-lane` uses, run in-process.
3. **Return.** A typed decision: **acquire** for A1, **refuse** for A2 until A1 releases. The refusal serializes the two writes; A2, re-run after A1's change is committed, sees R already cancelled and correctly does nothing. The clobber never lands.

The witness that scores it is the same kind as the gate's: the airline database hash, a byte the agents author zero of. A clobber is counted only when it is *directional* — the naive arrangement let the second agent apply a stale write the serialized second agent did not — which is the discipline that took the live headline from a reported 7 to an honest 6 (§6.5). The arbiter's refusal is a refusal in the identical sense the gate's block is: a typed "no" issued from a pure verdict over evidence the agent cannot forge. One reads the witness after the claim; the other prevents the collision before the write. Two moments, one machine.

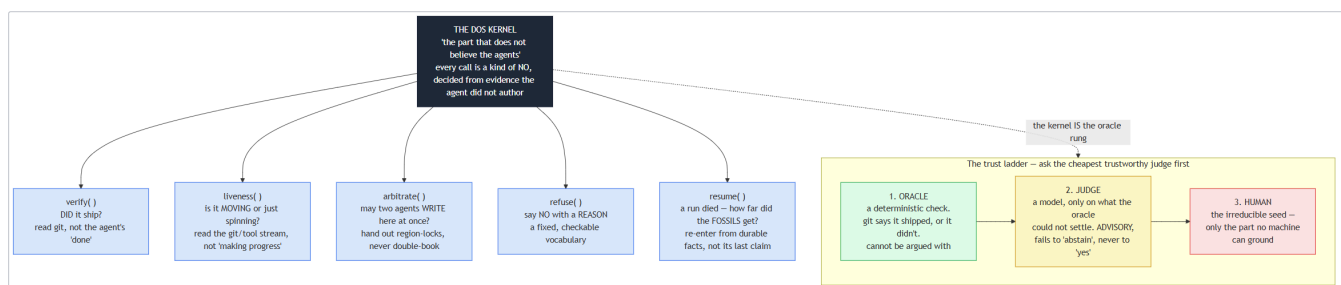


Figure B1. The same kernel map as Appendix A, now read as *machinery*. Each call is the B.1 pipeline with a different evidence source plugged into step 1: `verify` reads git, `arbitrate` reads the live leases, the write-admission gate reads the database hash. The branch they share is step 2 — every source is tagged by author — and step 3, the pure verdict, is the same closed-vocabulary "no" in every case. The trust ladder underneath is the accountability ladder of B.2 seen from the adjudicator side: the deterministic oracle decides first, a model-judge rules only on the residue and fails safe to abstain, a human grounds the irreducible seed.

B.5 Why the answer is a closed list, not a sentence

The fourth step — return a value from a fixed vocabulary — looks like a detail and is actually a load-bearing wall. A verdict that is free text ("looks mostly fine, but maybe double-check the booking") cannot be acted on by a machine and cannot be audited by a human: you cannot enumerate the prose a model might emit, so you cannot prove you handled every case. A verdict that is one of {admit, block, abstain} with a named reason can be branched on without parsing, logged without ambiguity, and — the part that matters for trust — *checked for completeness*: there are finitely many answers, so you can confirm none was forgotten and none overlaps. This is why the kernel's whole answer-space is a handful of closed enums, and why "abstain" is a first-class member rather than a shrug: a verdict that is allowed to say "I do not have a witness for this" is a verdict that never has to guess, and a kernel that never guesses is one whose every "block" you can believe.

That closure is also what lets the same verdict be spent in two places without re-deciding. The gate's "block" and the arbiter's "refuse" are the same typed object a downstream consumer reads; the consumer chooses what to do with it (drop the write, hold the lease, surface it to a human), but it never re-litigates the verdict. The decision was made once, purely, over unforgeable evidence — and after that it is just a value being routed. *That* is the mechanism behind the paper's one sentence: a verdict the agent cannot forge is harmful handed back

to the agent and valuable handed to anyone else, because "handed to anyone else" is nothing more than routing a closed, grounded value to a consumer who is not the one being judged.

How it works, in one line: read bytes at the boundary, tag each by its author, decide once with a pure function that only an unforgeable byte can sway, and emit one value from a closed list — then route that value anywhere except back to the agent it judged.

The companion engineering write-up — the gate, the arbiter, and the `effect_witness` floor with their tests — lives under `benchmark/agentprocessbench/writeadmit/` and `src/dos/effect_witness.py` in the DOS repository; the kernel's own architecture contract is `CLAUDE.md` and `docs/ARCHITECTURE.md`.

Appendix C. How to reproduce — every number, command, and released item

This appendix is the operator's manual for the paper. It lists, in one place, every result the body claims, the exact command that regenerates it, the artifact that backs it, and what you need to run it yourself. It is written so a stranger with the repository and a single API key can re-derive the headline numbers — or, where a number is gated behind spend or an external clone, see precisely what

stands between them and it. Two of the four results are **\$0 and deterministic** (they read a frozen, shipped CSV); two are **live and paid** (they drive an external benchmark through a model API). We mark each one, because the distinction is the whole methodological spine of the paper: a frozen replay gives a *rate*, only a live run gives a *payoff*.

C.1 The reproduction principle: re-derive, never trust the cache

One rule governs every script below, and it is the same rule the paper turns on, aimed at our own pipeline. **Every headline number is recomputed from the byte the agent could not author, never read back from a stored summary.** The frozen results recompute from a per-run CSV whose cells the benchmark's own grader wrote; the live results recompute each verdict from the stored message *with the current extractor*, not from the cached verdict bit. This is not pedantry: in the cross-model run (§6.4) a naive sum of the cached bits reported the stronger model at $J = 4$, and re-deriving from the stored message gave the true $J = 5$ — it recovered an over-claim the older extractor had missed. A cached result reads as clean unless you re-derive it. So each command below ends in a fold that re-reads ground truth, and several ship a `--check` mode that exits non-zero if the live data has drifted from the number the paper prints.

C.2 One-time setup

Everything runs from a single editable install of the kernel plus its benchmark extras. The kernel's own dependency set is PyYAML-only; the paid runs pull in the external benchmark and a model client.

Table C1. Setup, in order. Steps 1–2 suffice for the two \$0 results; steps 3–4 are needed only for the two live results.

#	Step	Command	Needed for
1	Clone + editable-install the kernel	<code>pip install -e .</code> (from the repo root)	everything
2	Confirm the suite is green	<code>python -m pytest -q</code>	everything
3	Clone + install tau2-bench (the live target; the PyPI tau2 is an unrelated squatter)	<code>pip install -e C:/work/tau2-bench</code> then <code>pip install audiop-its</code> on Python 3.13	§6 live runs
4	Put a model key in <code>.env</code> at the repo root	<code>GEMINI_API_KEY=...</code> (the runners load it from <code>.env</code> ; it is not read from the shell)	§6 live runs

Two setup traps cost real time and are worth stating once. `tau2-bench` transitively imports `audiop`, which was removed in Python 3.13 — install the `audiop-its` backport or the import fails before any task runs. And the Gemini key shipped as an access-token (an `AQ.`-prefixed string) *expires*; re-smoke it before any batch with a single `models?pageSize=1&key=...` request, or a dead key silently zeroes the run and makes “we tried, it found nothing” look like a result when it is an artifact.

C.3 The two \$0 results (frozen, deterministic, no key)

These read a shipped CSV and recompute the paper's numbers with no network, no model, no spend. They are the detector-quality story (§3 / Appendix A.5) and the give-up story (§4 / Appendix A.6). Each is its own single source of truth and ships a `--check` mode that fails loud on drift.

Table C2. The frozen results — reproduce with one command each, §

Paper claim	Command	Backing artifact
Detector precision 88–98%, recall ~2%, the trio-additivity +30% recall gain (§3, Tables 1–2)	<code>python -m benchmark.toolathlon.additivity (print) · --check (assert) · --emit (write the claims ledger)</code>	<code>benchmark/toolathlon/_res</code> — 7,116 frozen runs, 22 models : + detector
Give-up halts 0 winners at $K \geq 3$, ~11% tokens saved; the naive variant kills 6 (§4, Table 2)	<code>python benchmark/giveup_cross_benchmark.py (tables + figure) · --no-fig</code>	the same <code>replay_all_row benchmark/enterpriseops/1</code>
The frozen 13.6% over-claim slice (§6.2, the rate the live run promotes)	<code>python -m benchmark.agentprocessbench.overclaim --check → consensus=34 (13.6%) , exit 0</code>	the 250 frozen runs <code>AgentProcess (dataset.load(conf</code>

The durable-rows CSV carries a fingerprint, and the paper records it: the rows this paper's numbers were drawn from fingerprint to `1a55e8d8e2d4 (additivity.rows.fingerprint())`. The paper's own build step warns if the live CSV has drifted from that value, so a stale paper cannot quietly ship wrong provenance. If you regenerate the rows and the fingerprint moves, the numbers move with it — that is the point.

C.4 The two live results (paid, gated behind a key)

These are the headline of the paper — the out-of-loop payoff, run live (§6). They drive a real agent through `tau2-bench`, read the environment's own database hash, and count an integer J of corruptions a referee actually prevented. Both the agent and the user-simulator are live models, so cost scales with dialogue length; the whole program in the paper ran for well under the budgets below. Every live-output directory is **gitignored on purpose** — the seed configs embed the API key — so the repository ships the *drivers* that regenerate the result, not the raw transcripts. The \$0 synthetic smoke that each driver ships proves the mechanism with hand-built tool-calls and no model; run it first, every time.

Table C3. The live results. Run the \$0 smoke first; the paid command reproduces the paper's J

Paper claim	\$0 smoke (mechanism)	Paid run
Write-admission, single model: $J = 5$, 11.6% over-claim rate (§6.3, Fig 8)	<code>python -m benchmark.agentprocessbench.writeadmit.live_loop with frozen_ab (no key) — J arithmetic proven</code>	<code>benchm</code>
Cross-model: $J = 10$ over 120 clean tasks, 8.3% on both flash and pro (§6.4, Fig 8b)	(same gate; the smoke above covers it)	<code>run _t</code> once <code>benchma</code>
Coordination: $J = 6$, arbiter serialized 8/8 pairs (§6.5, Fig 9)	<code>python benchmark/agentprocessbench/writeadmit/coord_loop.py --smoke — the arbiter-serialization invariant, no key</code>	<code>benchm</code>

Two model-specific details are load-bearing and easy to miss, because each one silently zeroes a run. First: the cross-model run needs a **model-aware reasoning setting**. The flash tier needs a flag that suppresses a flash-only

crash on long dialogues; the *same* flag is fatal to gemini-2.5-pro, which is thinking-only and rejects it (“*Budget 0 is invalid — this model only works in thinking mode*”). A first pro attempt that re-used the flash flag errored on all 60 tasks and reported a false $J = 0$; the correct setting is per-tier, and a regression test (`test_model_args.py`) now pins it so the artifact cannot recur. Second: the coordination run must give each task the reservation's **real owner** as the user identity, or a correct agent rightly refuses to touch a stranger's booking and the designed conflict never fires. Neither is a kernel bug; both are the harness details that make “we tried, it found nothing” an artifact rather than a result, which is why the paper names them.

The complete, append-only run journals for the live program — every checkpoint, every trap, every dollar — live next to the drivers and are the most faithful record of how the numbers were produced: `benchmark/agentprocessbench/writeadmit/RUN_PLAN_live.md` (the write-admission run, $J = 5 \rightarrow$ the design notes that became `docs/228`), and the matching design notes `docs/232` (cross-model, $J = 10$) and `docs/233` (coordination, $J = 6$).

C.5 The full set of released items

The paper rests on a kernel, a benchmark suite, a set of design notes, and the paper's own build. This is the complete inventory, organised by what each thing is for. The dependency arrow is one-way throughout: the benchmarks and the paper `import dos` / call the `dos` CLI; nothing under `src/dos/` depends on them.

C.5.1 The kernel (the shipped product)

Table C4. The DOS kernel — the deterministic referee the paper measures. Distribution `dos-kernel v0.13.0`; import name and CLI are `dos`.

Item	Where	What it is
The syscall kernel	<code>src/dos/</code>	the pure verdicts: <code>verify</code> (did it ship?), <code>liveness</code> (is it moving?), <code>arbitrate</code> (may two agents write here?), <code>refuse</code> (a typed “no”), <code>resume</code> (re-enter a dead run) — each a pure <code>classify(evidence, policy)</code>
The write-admission floor	<code>src/dos/effect_witness.py</code>	the <code>believe_under_floor</code> discipline both live consumers stand on: an agent-authored read-back yields <i>unwitnessed</i> , never <i>refuted</i> (Appendix B.2)
The CLI + read-only projections	<code>src/dos/cli.py</code> , <code>dispatch_top*.py</code> , <code>decisions*.py</code>	the <code>dos</code> verbs; the <i>what-needs-me</i> and <i>what's-running</i> screens (behind the <code>[tui]</code> extra)
The MCP server	<code>src/dos_mcp/</code> (the <code>[mcp]</code> extra)	the syscalls as MCP tools — JSON over stdio, no <code>import dos</code> required of the consumer
The kernel test suite	<code>tests/</code>	the litmus tests that pin the layering (kernel imports no host, no vendor, no plan) — must stay green
The architecture contract	<code>CLAUDE.md</code> , <code>docs/ARCHITECTURE.md</code>	the four-layer rule and the per-module byte-clean argument

C.5.2 The benchmark suite (the evidence)

All seven programs share one standardized runner, so the inventory, prerequisites, and estimated cost of every benchmark are one command away: `python -m benchmark._run list`. The three the paper draws on are marked.

Table C5. The released benchmarks. ★ = a paper result is drawn from it.

Benchmark	Where	What it measures
toolathlon ★	<code>benchmark/toolathlon/</code>	the frozen 22-model replay: detector precision/recall/additivity (§3) and the give-up zero-cost claim (§4)
agentprocessbench ★	<code>benchmark/agentprocessbench/</code>	the over-claim slice (§6.2) and the live write-admission + coordination payoffs under <code>writeadmit/</code> (§6.3–6.5)
enterpriseops ★	<code>benchmark/enterpriseops/</code>	the second give-up corpus and the in-loop intervention bake-off (§4.3 / Appendix A.6)
<code>agentdiff</code>	<code>benchmark/agentdiff/</code>	a state-diff write-admission target (the live ΔB driver; <code>docs/237</code>)
<code>fleet_horizon</code>	<code>benchmark/fleet_horizon/</code>	the believed-vs-adjudicated fleet A/B over a long horizon
<code>fleetforge</code>	<code>benchmark/fleetforge/</code>	the fleet-coordination scaffold
<code>agenthallu</code>	<code>benchmark/agenthallu/</code>	the structural + unrecovered-error detector that cut false-alarms 29×

C.5.3 The paper and its figures (reproducible build)

Table C6. The paper build. Every figure regenerates from the run data; the prose, sections, and registry are single-sourced.

Item	Where	Note
The build	<code>python paper/build.py</code>	refresh figures $\rightarrow$ assemble HTML render PDF; warns on rows-finger drift
The single source of truth	<code>paper/meta.py</code>	title, section order (auto-discovery), <code>NN_</code> prefix, figure registry, the fingerprint
The prose	<code>paper/sections/*.html</code>	one fragment per section; a new fragment is picked up automatically
The figure generators	<code>paper/figs_src/*.py</code>	the matplotlib scripts that turn the data into each payoff figure (e.g. <code>payoff_writeadmit_crossmodel_coord_payoff_live.py</code>)
The plain-language companions	<code>benchmark/toolathlon/EXPLAINER.md</code> , <code>GLOSSARY.md</code>	the engineering twin of Appendix

C.6 The reproduction checklist, in one screen

If you do nothing else, this is the shortest path from a clean clone to the paper's numbers. The first three lines need no key and finish in seconds; the rest need a key and a tau2 clone and cost a few dollars.

Table C7. The whole reproduction, top to bottom.

#	Do this	Get
1	<code>pip install -e . && python -m pytest -q</code>	a green kernel — the referee itself
2	<code>python -m benchmark.toolathlon.additivity --check</code>	the detector-quality numbers (§3), \$0
3	<code>python benchmark/giveup_cross_benchmark.py --no-fig</code>	the give-up zero-cost result (§4), \$0
4	<code>install tau2-bench + set GEMINI_API_KEY</code> ; re-smoke the key	the live target is reachable
5	<code>coord_loop.py --smoke then --live --pairs 8</code>	the coordination payoff $J = 6$ (§6.5)
6	<code>_run_model.py</code> for flash and for pro, then <code>aggregate_live</code>	the write-admission payoff $J = 10$ (§6.3–6.4)

The reproduction, in one line: two commands recompute the frozen rate from a shipped CSV at \$0; two more drive a live agent through tau2-bench off a single key to recompute the payoff — and every one of them re-derives its number from the byte the agent could not author, never from a cached summary.